并行计算

——（1）结构

讲授：鄂金龙

课程介绍



- 教师—鄂金龙

  毕业于清华大学计算机系，人大信息学院副教授

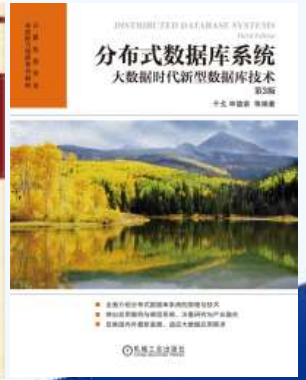
  研究方向：云/边缘计算、物联网、流媒体技术等

  邮箱：ejinlong@ruc.edu.cn　办公室：信息楼125B



- 参考教材
  - 并行计算——结构、算法、编程（第3版），高等教育出版社，陈国良编著
  - 数据密集型应用系统设计[DDIA]，中国电力出版社，Martin Kleppmann著
  - 分布式数据库系统-大数据时代新型数据库技术（第2/3版），机械工业出版社，于戈、申德荣等编著





- 上课考试安排

  6.6/7/14 9:00-12:00 13:30-16:30，7月初考试

1.并行计算机结构模型

并行计算

- 并行计算：并行机上所作的计算，又称高性能计算或超级计算。
- 计算科学：计算物理、计算化学、计算生物等
- 科学与工程问题的需求：气象预报、油藏模拟、核武器数值模拟、航天器设计、基因测序等。
- 需求类型：计算密集、数据密集、网络密集。
- 美国HPCC计划：重大挑战性课题，3T性能
- 美国Petaflops研究项目：Pflop/s。
- 美国ASCI计划：核武器数值模拟。

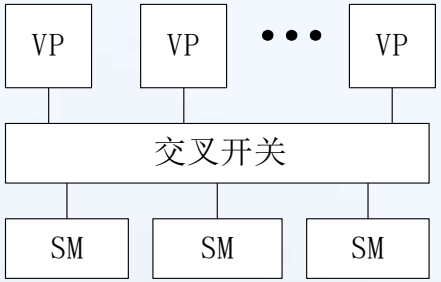
高性能计算机（早期）

- Intel（Option Red):
  1Tflops,1997,Pentium Pro
- SGI(Option Blue Mountain):
  3Tflops,1998,MIPS10000
- IBM(Option White):
  7Tflops,Top4,2001,Power3
- 日本Earth Simulator:
  35Tflops,Top1,2002,VP
- Hewlett-Packard ASCI Q:
  7Tflops ,Top2,3,2002, Alpha Server
- 中国联想:
  1Tflops,Top43,2002

高性能计算机（最新榜单）

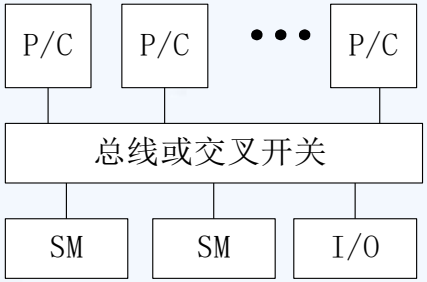
序号	系统	所属国家	内核	运算性能 (PFlop/s)	峰值性能 (PFlop/s)	功率 (千瓦)
1	El Capitan	美国	11340000	1809.00	2821.10	29685
2	Frontier	美国	9066176	1353.00	2055.72	24607
3	Aurora	美国	9264128	1012.00	1980.01	38698
4	JUPITER Booster	德国	4801344	1000.00	1226.28	15794
5	Eagle	美国	2073600	561.20	846.84	/
6	HPC6	意大利	3143520	477.90	606.97	8461
7	Supercomputer Fugaku	日本	7630848	442.01	537.21	29899
8	Alps	瑞士	2121600	434.90	574.84	7124
9	LUMI	芬兰	2752704	379.70	531.51	7107
10	Leonardo	意大利	1824768	241.20	306.31	7494

并行计算机结构模型



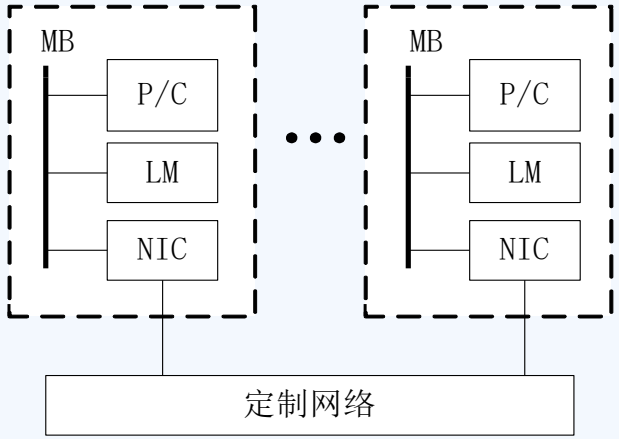
(a) PVP

并行向量处理器



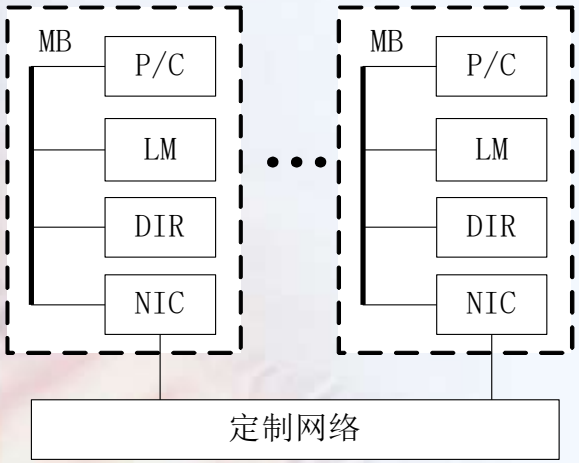
(b) SMP

对称多处理机



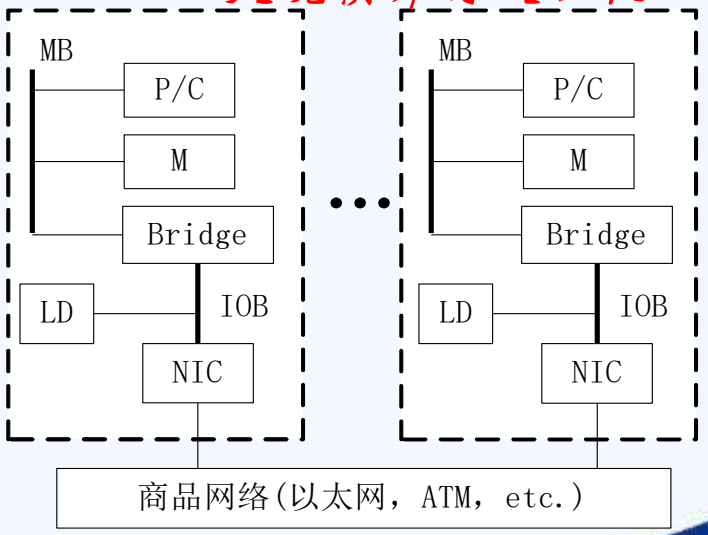
(c) MPP

大规模并行处理机



(d) DSM

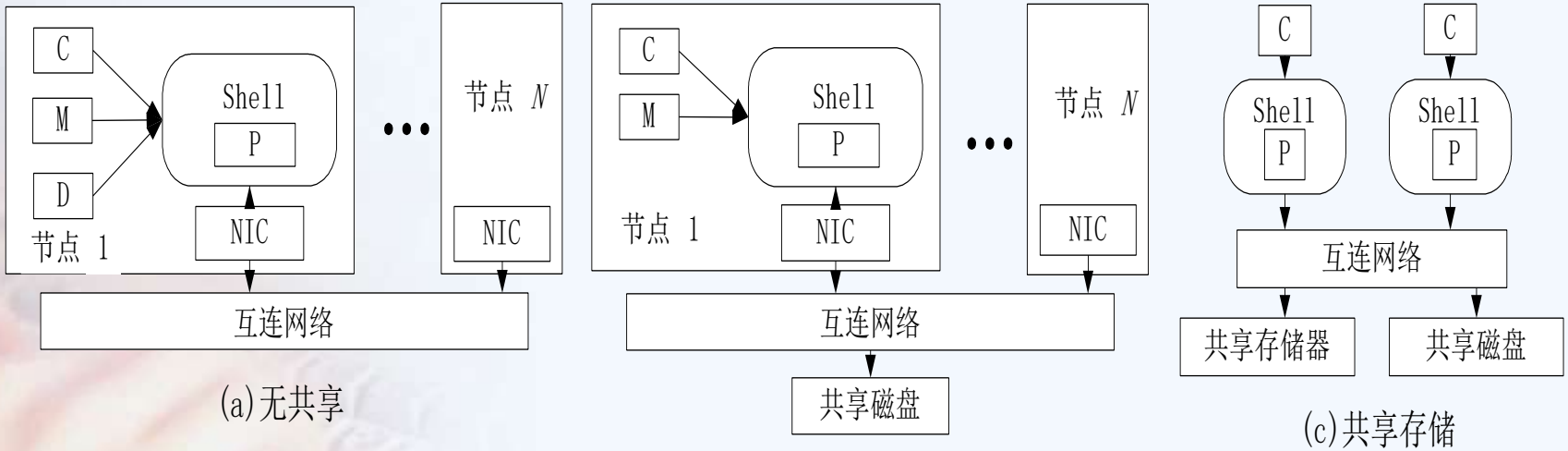
分布共享存储处理机



工作站机群 (e) COW

并行计算机体系合一结构

- SMP、MPP、DSM和COW并行结构渐趋一致。
  - 大量的节点通过高速网络互连起来
  - 节点遵循Shell结构：用专门定制的Shell电路将商用微处理器和节点的其它部分（包括板级Cache、局存、NIC和DISK）连接起来。优点是CPU升级只需要更换Shell。



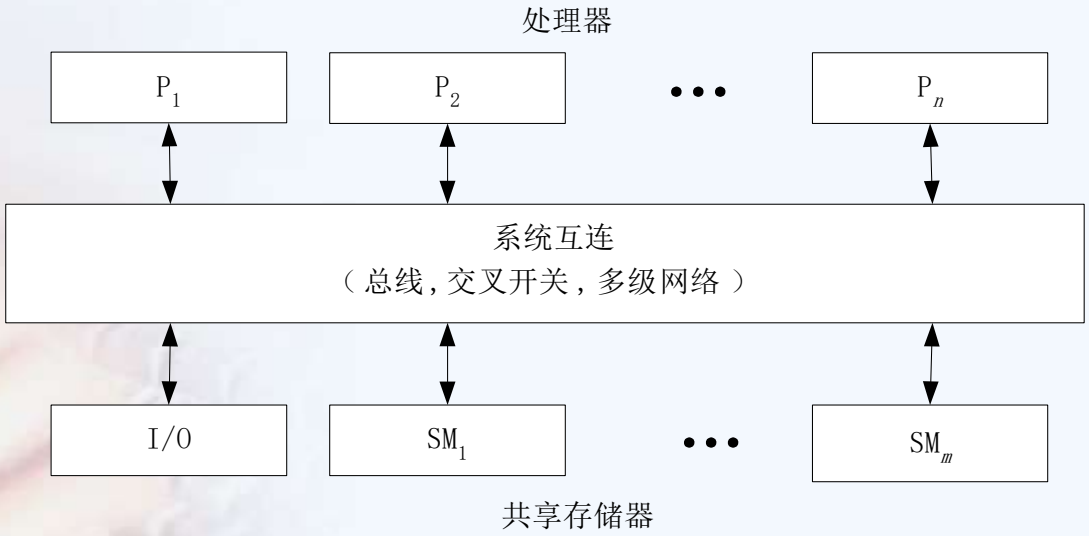
(a)无共享　(b)共享磁盘　(c)共享存储

五种结构特性一览表

属性	PVP	SMP	MPP	DSM	COW
结构类型	MIMD	MIMD	MIMD	MIMD	MIMD
处理器类型	专用定制	商用	商用	商用	商用
互连网络	定制交叉开关	总线、交叉开关	定制网络	定制网络	商用网络（以太ATM）
通信机制	共享变量	共享变量	消息传递	共享变量	消息传递
地址空间	单地址空间	单地址空间	多地址空间	单地址空间	多地址空间
系统存储器	集中共享	集中共享	分布非共享	分布共享	分布非共享
访存模型	UMA	UMA	NORMA	NUMA	NORMA
代表机器	Cray C-90，Cray T-90，银河1号	IBM R50，SGI Power Challenge，曙光1号	Intel Paragon，IBMSP2，曙光1000/2000	Stanford DASH，Cray T 3D	Berkeley NOW，Alpha Farm

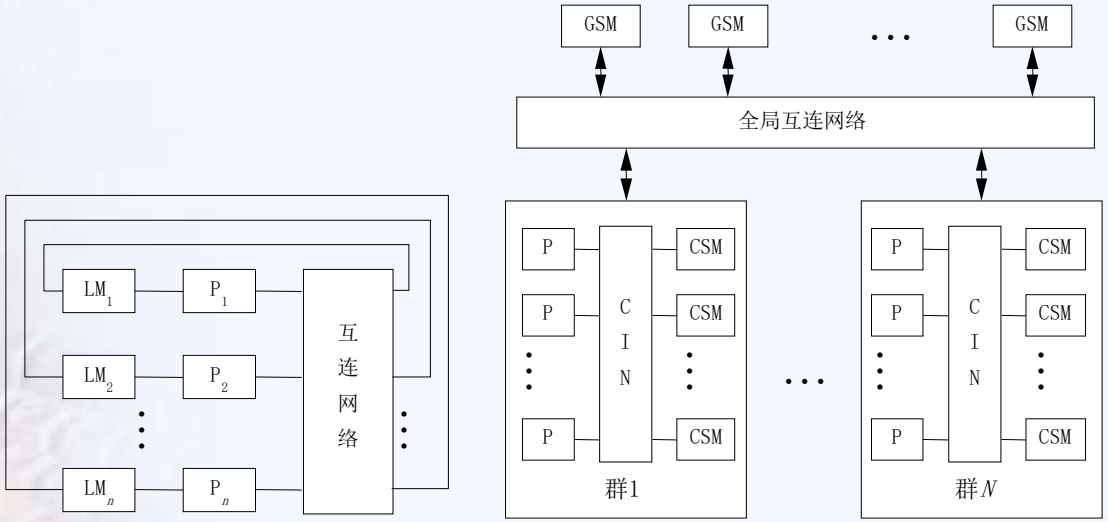
并行计算机访存模型（1）

- UMA（Uniform Memory Access）模型是均匀存储访问模型的简称。其特点是：
  - 物理存储器被所有处理器均匀共享；
  - 所有处理器访问任何存储字取相同的时间；
  - 每台处理器可带私有高速缓存；
  - 外围设备也可以一定形式共享。



并行计算机访存模型（2）

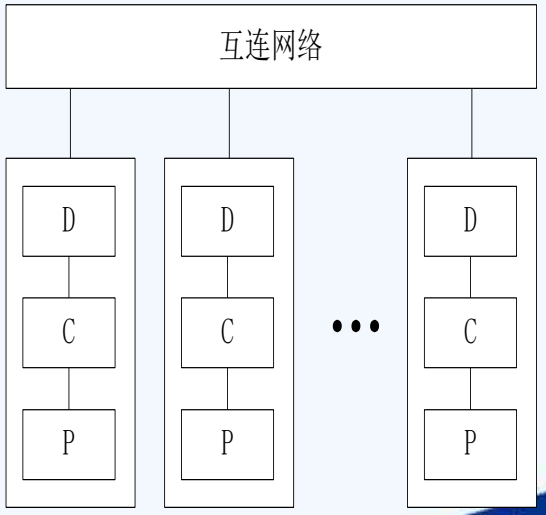
- NUMA(Nonuniform Memory Access)模型是**非均匀存储访问**模型的简称。特点是：
  - 被共享的存储器在物理上是分布在所有的处理器中的，其所有本地存储器的集合就组成了全局地址空间；
  - 处理器访问存储器的时间是不一样的；访问本地存储器LM或群内共享存储器CSM较快，而访问外地的存储器或全局共享存储器GSM较慢(此即非均匀存储访问名称的由来)；
  - 每台处理器照例可带私有高速缓存，外设也可以某种形式共享。



(a) 共享本地存储模型　　(b) 层次式机群模型

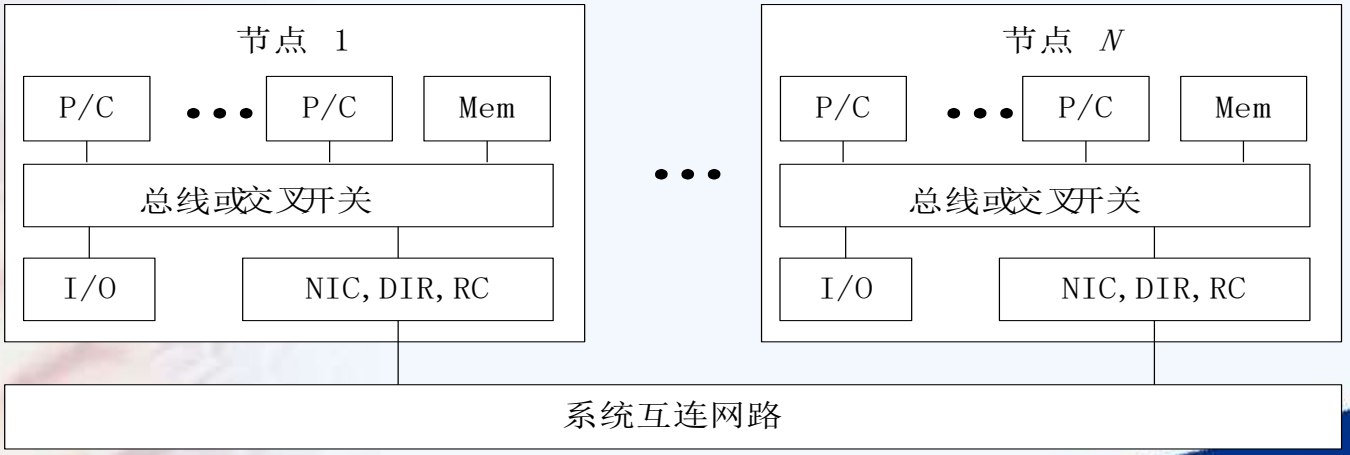
并行计算机访存模型（3）

- COMA(Cache-Only Memory Access)模型是**全高速缓存存储访问**的简称。其特点是：
  - 各处理器节点中没有存储层次结构，全部高速缓存组成了全局地址空间；
  - 利用分布的高速缓存目录D进行远程高速缓存的访问；
  - COMA中的高速缓存容量一般都大于2 级高速缓存容量；
  - 使用COMA时，数据开始时可任意分配，因为在运行时它最终会被迁移到要用到它们的地方。



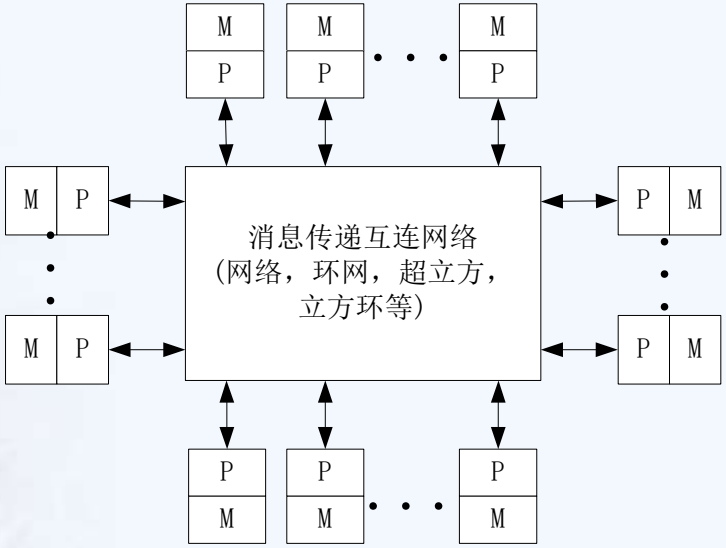
并行计算机访存模型（4）

- CC-NUMA（Coherent-Cache Nonuniform Memory Access）模型是**高速缓存一致性非均匀存储访问**模型的简称。其特点是：
  - 大多数使用基于目录的高速缓存一致性协议；
  - 保留SMP结构易于编程的优点，也改善常规SMP的可扩放性；
  - CC-NUMA实际上是一个分布共享存储的DSM多处理机系统；
  - 它最显著的优点是程序员无需明确地在节点上分配数据，系统的硬件和软件开始时自动在各节点分配数据，在运行期间，高速缓存一致性硬件会自动地将数据迁移至要用到它的地方。

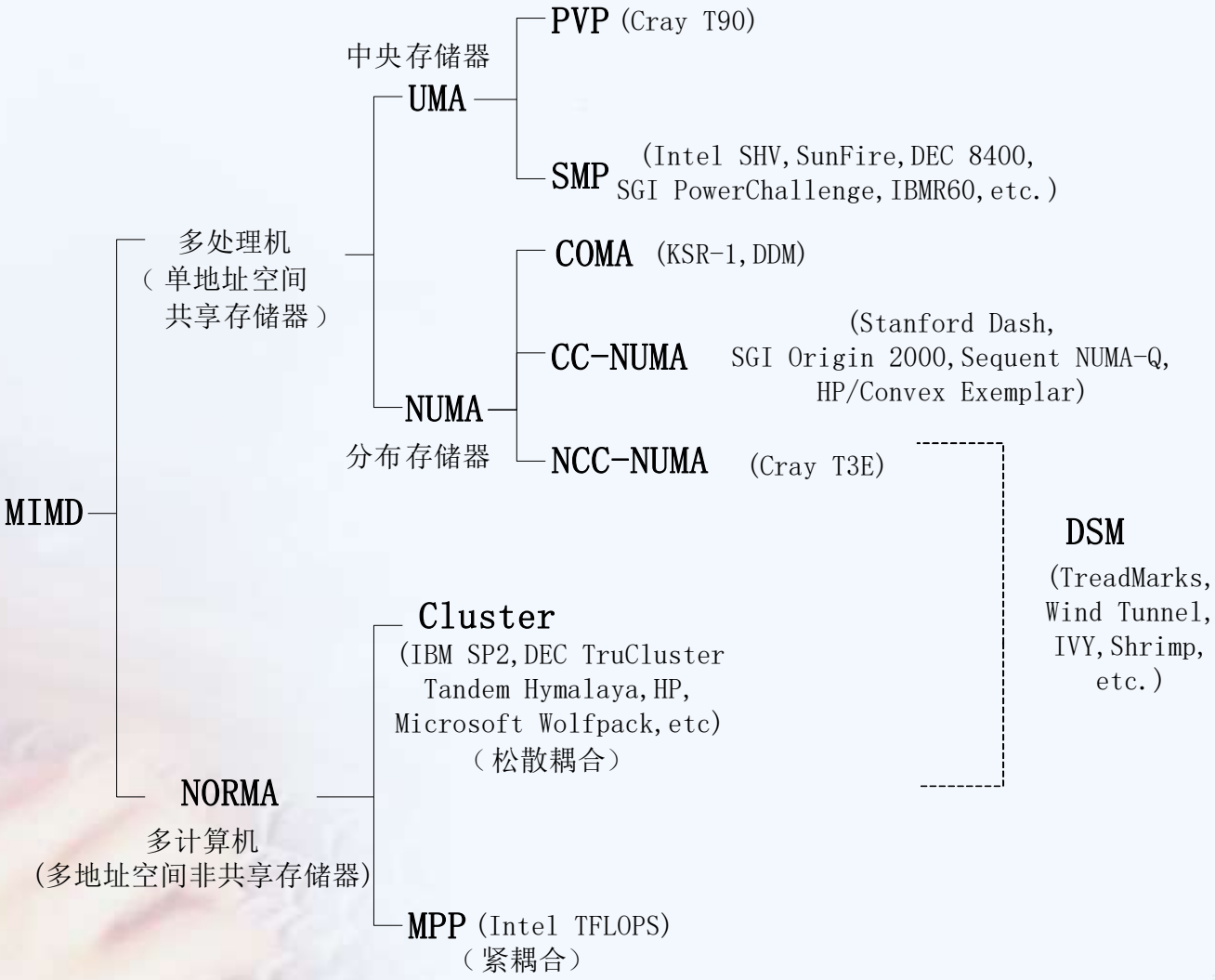


并行计算机访存模型（5）

- NORMA（No-Remote Memory Access）模型是**非远程存储访问**模型的简称。NORMA的特点是：
  - 所有存储器是私有的；
  - 绝大数NUMA都不支持远程存储器的访问；
  - 在DSM中，NORMA就消失了。

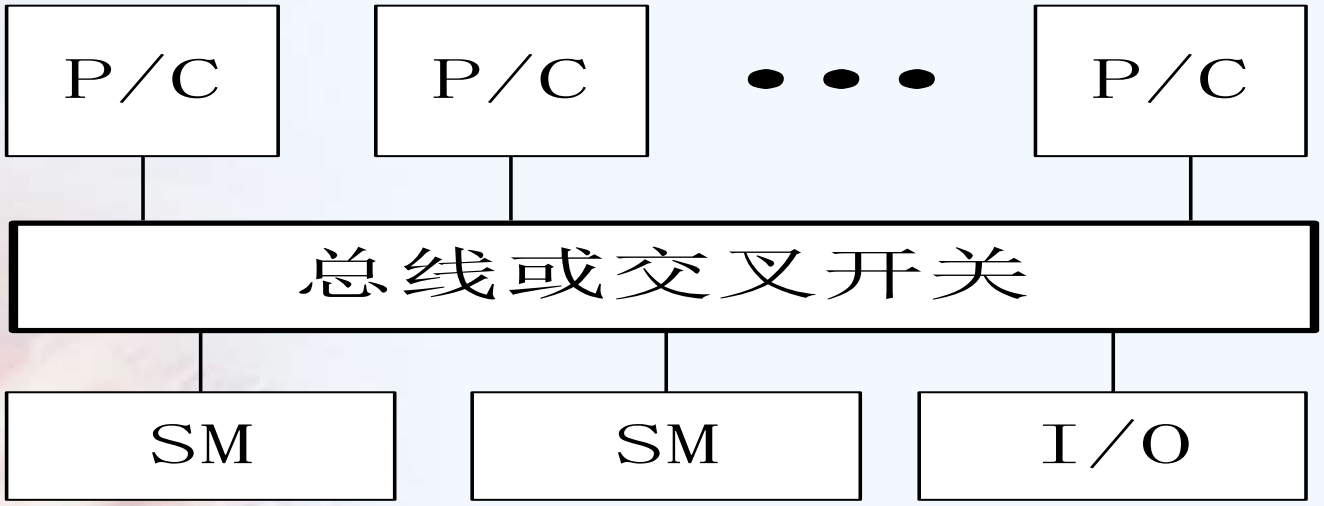


构筑并行机系统的不同存储结构



对称多处理机SMP(1)

- SMP: 采用商用微处理器，通常有片上和片外Cache，基于总线连接，集中式共享存储，UMA结构
- 例子：SGI Power Challenge, DEC Alpha Server,Dawning 1



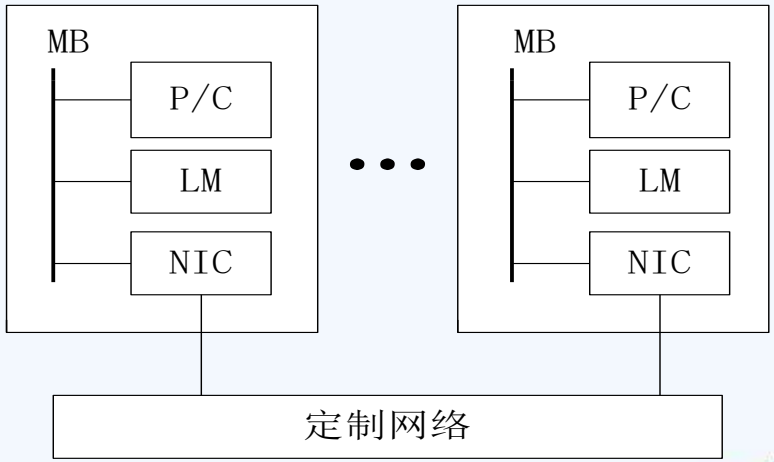
对称多处理机SMP(2)

- 优点
  - 对称性
  - 单地址空间，易编程性，动态负载平衡，无需显示数据分配
  - 高速缓存及其一致性，数据局部性，硬件维持一致性
  - 低通信延迟，Load/Store完成
- 问题
  - 欠可靠，BUS,OS,SM
  - 通信延迟（相对于CPU），竞争加剧
  - 慢速增加的带宽（MB double/3年,IOB更慢）
  - 不可扩放性---〉CC-NUMA

大规模并行机MPP

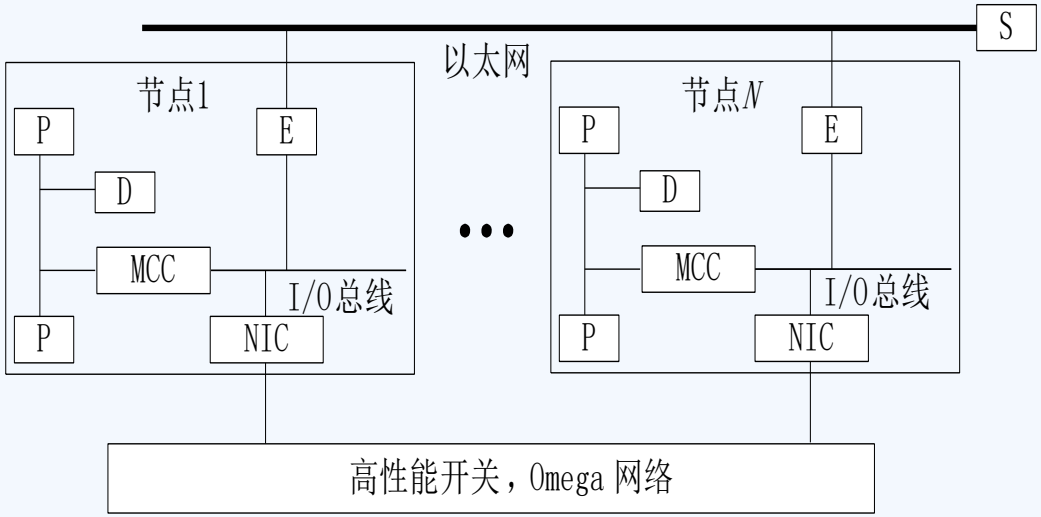
- 成百上千个处理器组成的大规模计算机系统，规模是变化的。
- NORMA结构，高带宽低延迟定制互连。
- 可扩放性：Mem, I/O,平衡设计
- 系统成本：商用处理器，相对稳定的结构，SMP,分布
- 通用性和可用性：不同的应用，PVM,MPI,交互，批处理，互连对用户透明，单一系统映象，故障
- 通信要求
- 存储器和I/O能力
- 例子：Intel Option Red

  IBM SP2 Dawning 1000



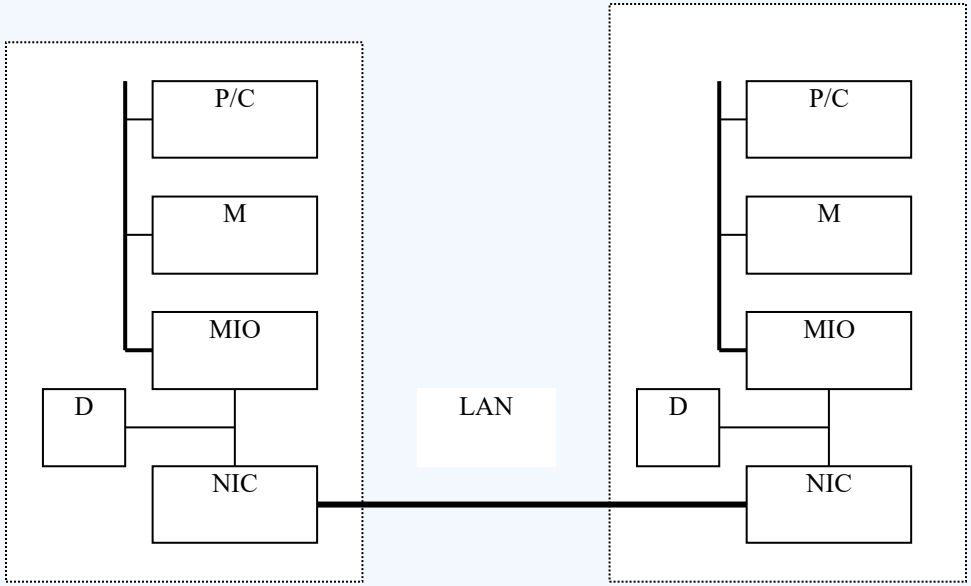
机群型大规模并行机SP2

- 设计策略：
  - 机群体系结构
  - 标准环境
  - 标准编程模型
  - 系统可用性
  - 精选的单一系统映像
- 系统结构：
  - 高性能开关 HPS 多级Ω网络
  - 宽节点、窄节点和窄节点2



工作站机群COW

- 分布式存储，MIMD，工作站+商用互连网络，每个节点是一个完整的计算机，有自己的磁盘和操作系统，而MPP中只有微内核
- 优点：
  - 投资风险小
  - 系统结构灵活
  - 性能/价格比高
  - 能充分利用分散的计算资源
  - 可扩放性好
- 问题
  - 通信性能
  - 并行编程环境



- 例子：Berkeley NOW，Alpha Farm, FXCOW

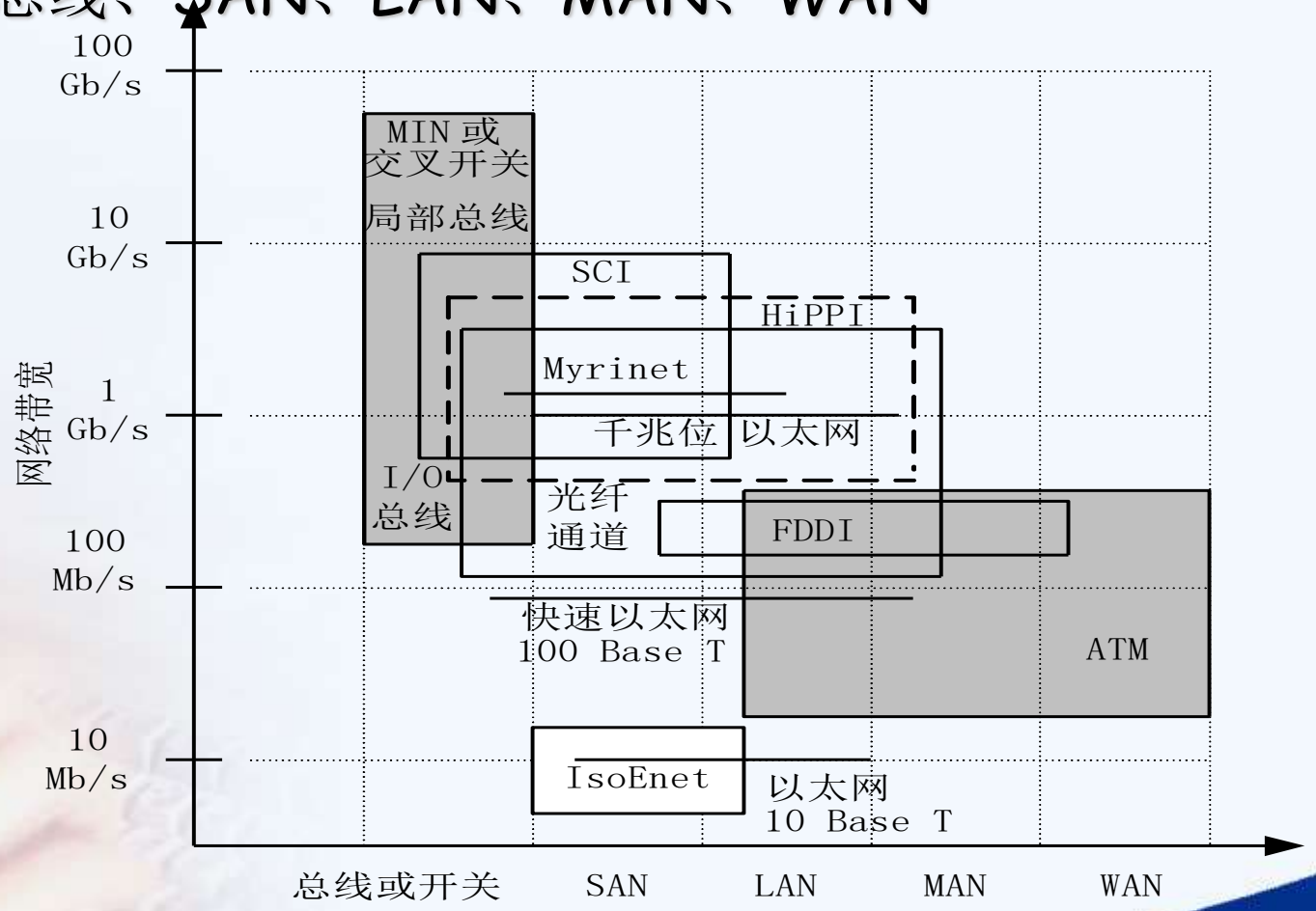
SMP\MPP\机群比较

系统特征	SMP	MPP	机群
节点数量(N)	≤O(10)	O(100)-O(1000)	≤O(100)
节点复杂度	中粒度或细粒度	细粒度或中粒度	中粒度或粗粒度
节点间通信	共享存储器	消息传递 或共享变量（有DSM时）	消息传递
节点操作系统	1	N(微内核) 和1个主机OS(单一)	N (希望为同构)
支持单一系统映像	永远	部分	希望
地址空间	单一	多或单一（有DSM时）	多个
作业调度	单一运行队列	主机上单一运行队列	协作多队列
网络协议	非标准	非标准	标准或非标准
可用性	通常较低	低到中	高可用或容错
性能/价格比	一般	一般	高
互连网络	总线/交叉开关	定制	商用

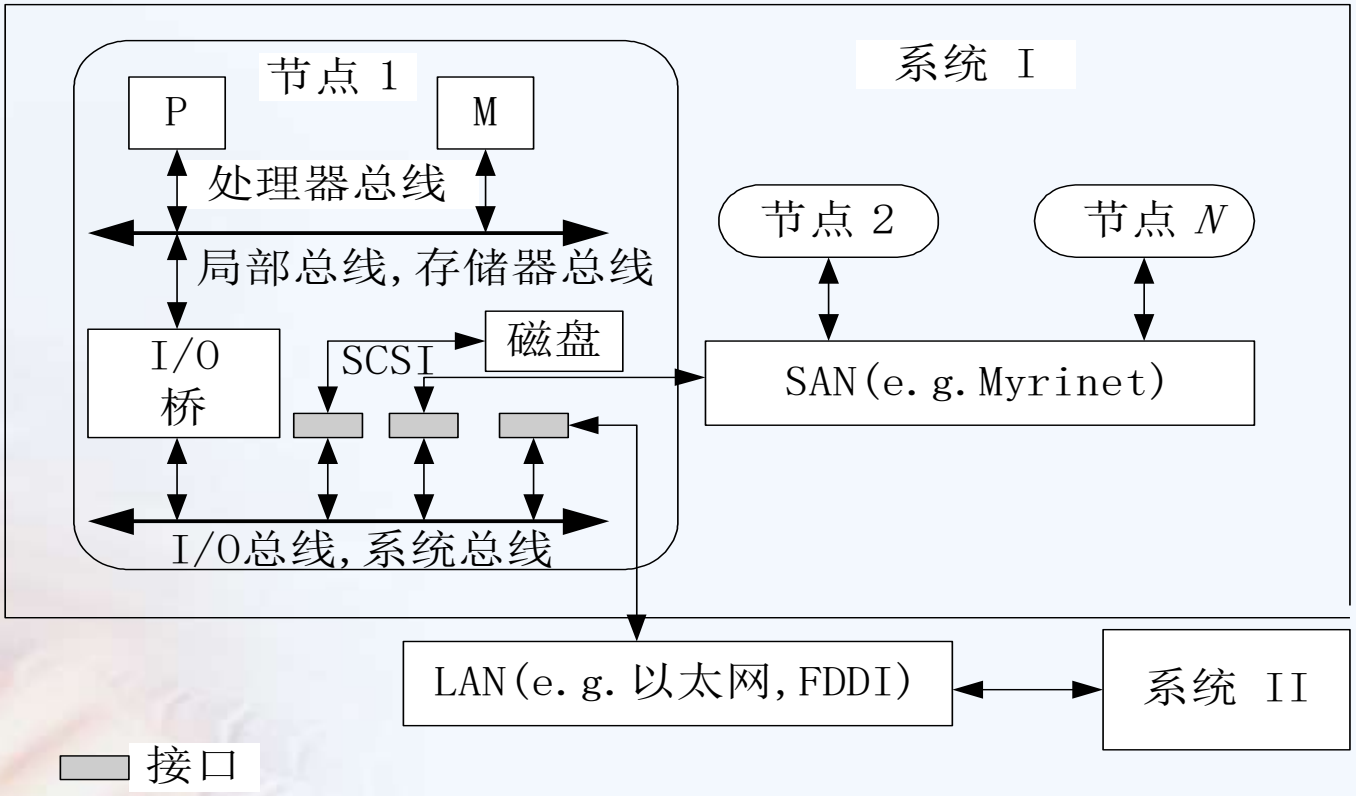
2.并行计算机互联与通信

系统互连

- 不同带宽与距离的互连技术:
  总线、SAN、LAN、MAN、WAN



局部总线、I/O总线、SAN和LAN



静态互连网络 与动态互连网络

- 静态互连网络：处理单元间有着固定连接的一类网络，在程序执行期间，这种点到点的连接保持不变；典型的静态网络有一维线性阵列、二维网孔、树连接、超立方网络、立方环、洗牌交换网、蝶形网络等
- 动态网络：用交换开关构成的，可按应用程序的要求动态地改变连接组态；典型的动态网络包括总线、交叉开关和多级互连网络等。

网络性能指标

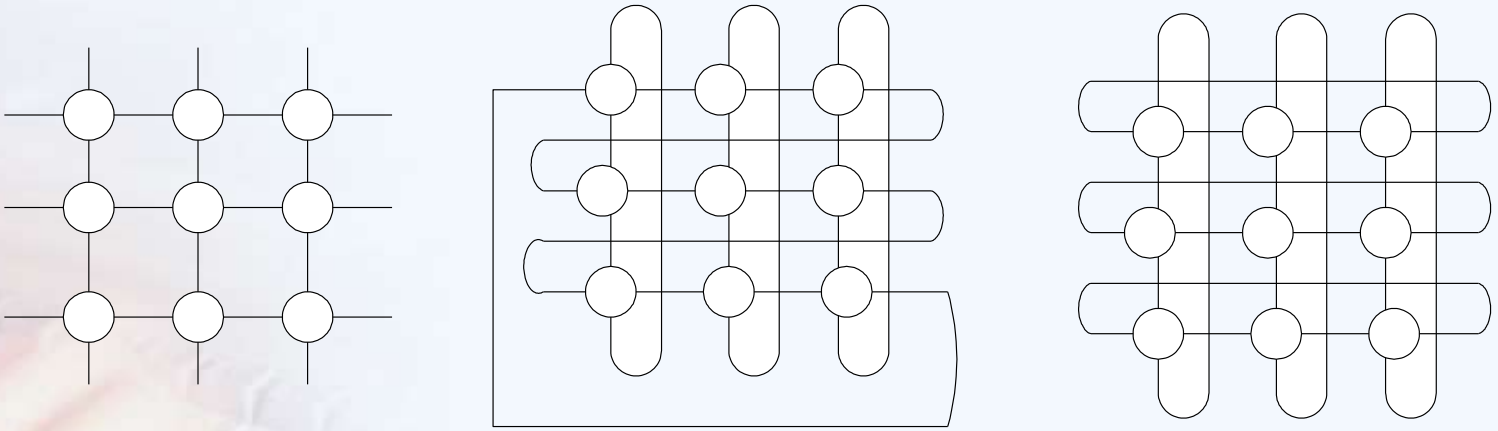
- 节点度（Node Degree）：射入或射出一个节点的边数。在单向网络中，入射和出射边之和称为节点度。
- 网络直径（Network Diameter）： 网络中任何两个节点之间的最长距离，即最大路径数。
- 对剖宽度（Bisection Width）：对分网络各半所必须移去的最少边数（*允许两边差1行*）
- *对剖带宽（Bisection Bandwidth）:每秒钟内，在最小的对剖平面上通过所有连线的最大信息位（或字节）数*
- 如果从任一节点观看网络都一样，则称网络为对称的（Symmetry）

静态互连网络（1）

- 一维线性阵列（1-D Linear Array）：
  - 并行机中最简单、最基本的互连方式，
  - 每个节点只与其左、右近邻相连，也叫二近邻连接，
  - N个节点用N-1条边串接之，内节点度为2，直径为N-1，对剖宽度为1
  - 当首、尾节点相连时可构成循环移位器，在拓扑结构上等同于环，环可以是单向的或双向的，其节点度恒为2，直径或为 $\lfloor N/2 \rfloor$（双向环）或为N-1（单向环），对剖宽度为2

静态互连网络（2）

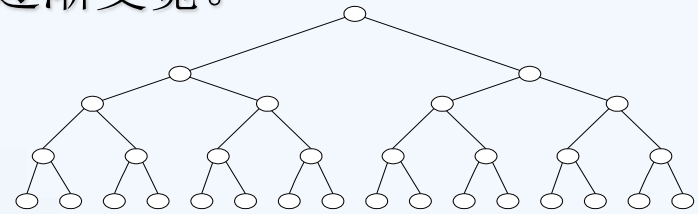
- $\sqrt{N} \times \sqrt{N}$ 二维网孔（2-D Mesh）：
  - 每个节点只与其上、下、左、右的近邻相连（边界节点除外），节点度为4，网络直径为 $2(\sqrt{N}-1)$ ，对剖宽度为 $\sqrt{N}$
  - 在垂直方向上带环绕，水平方向呈蛇状，就变成Illiac网孔了，节点度恒为4，网络直径为 $\sqrt{N}-1$ ，而对剖宽度为 $2\sqrt{N}$
  - 垂直和水平方向均带环绕，则变成了2-D环绕（2-D Torus），节点度恒为4，网络直径为 $2\lfloor \sqrt{N}/2 \rfloor$ ，对剖宽度为 $2\sqrt{N}$



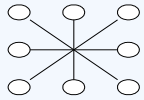
(a)2-D网孔　(b)Illiac网孔　(c)2-D环绕

静态互连网络（3）

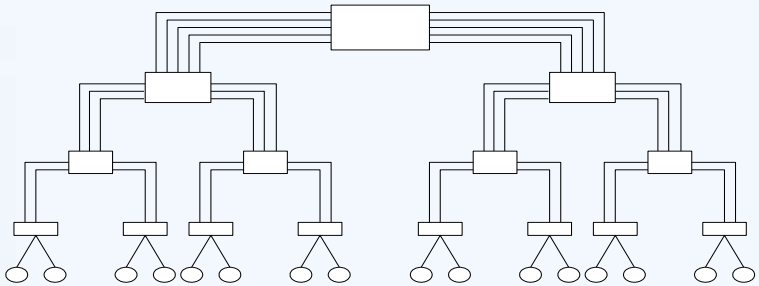
- 二叉树：
  - 除了根、叶节点，每个内节点只与其父节点和两个子节点相连。
  - 节点度为3，对剖宽度为1，而树的直径为 $2(\lceil \log N \rceil - 1)$
  - 如果尽量增大节点度为N-1，则直径缩小为2，此时就变成了星形网络，其对剖宽度为 $\lfloor N/2 \rfloor$
  - 传统二叉树的主要问题是根易成为通信瓶颈。胖树节点间的通路自叶向根逐渐变宽。



(a) 二叉树



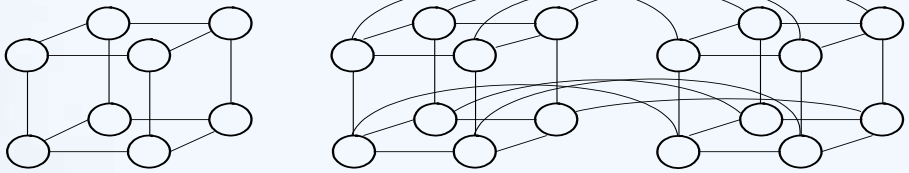
(b) 星形连接



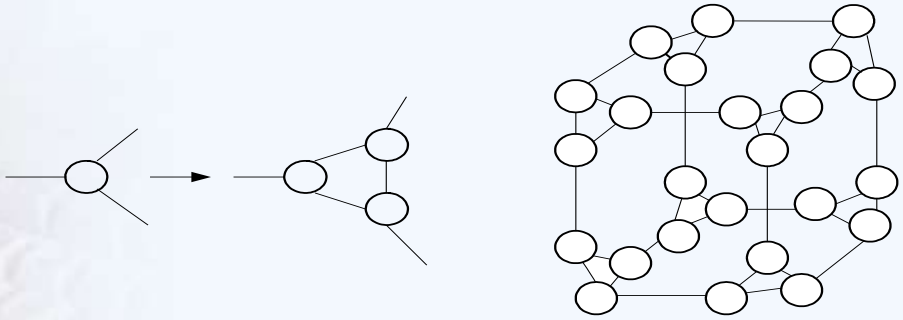
(c) 二叉胖树

静态互连网络（4）

- 超立方：
  - 一个n-立方由 $N = 2^n$ 个顶点组成，3-立方如图(a)所示；4-立方如图(b)所示，由两个3-立方的对应顶点连接而成。
  - n-立方的节点度为n，网络直径也是n，而对剖宽度为$N/2$。
  - 如果将3-立方的每个顶点代之以一个环就构成了如图(d)所示的3-立方环，此时每个顶点的度为3，而不像超立方那样节点度为n。



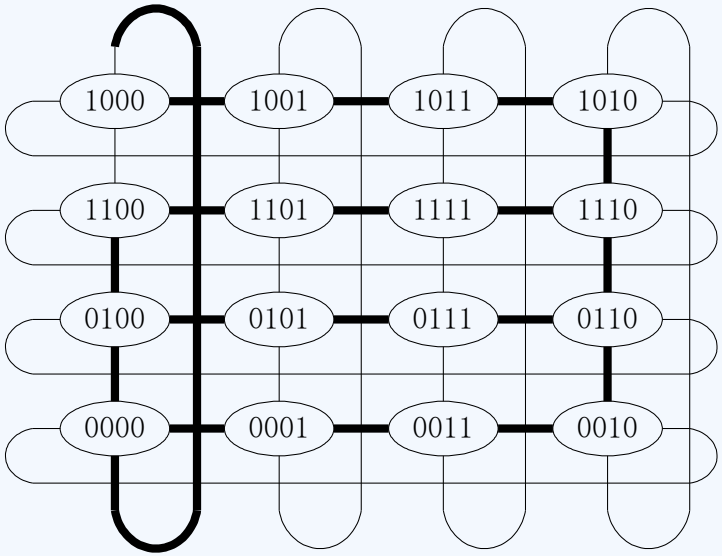
(a) 3−立方　　(b) 4−立方



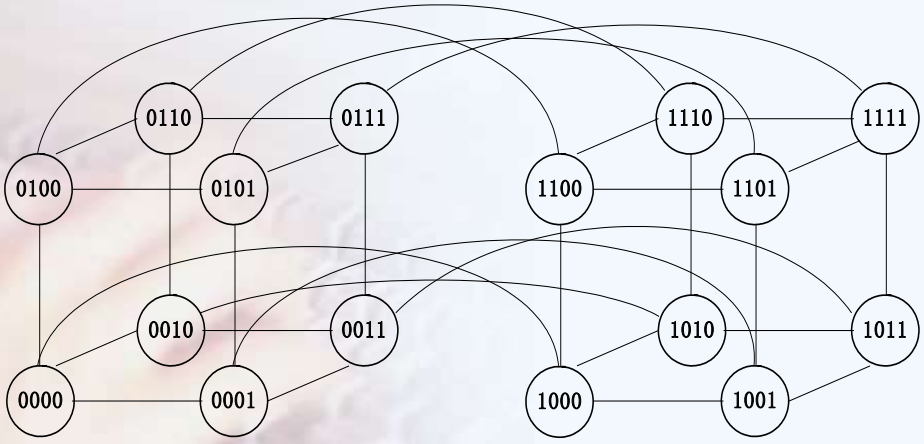
(c) 顶点代之以环　　(d) 3−立方环

嵌入

- 将网络中的各节点映射到另一个网络中去
- 用膨胀（Dilation）系数来描述嵌入的质量，它是指被嵌入网络中的一条链路在所要嵌入的网络中对应所需的最大链路数
- 如果该系数为1，则称为完美嵌入。



- 环网可完美嵌入到2-D环绕网中
- 超立方网可完美嵌入到2－D环绕网中

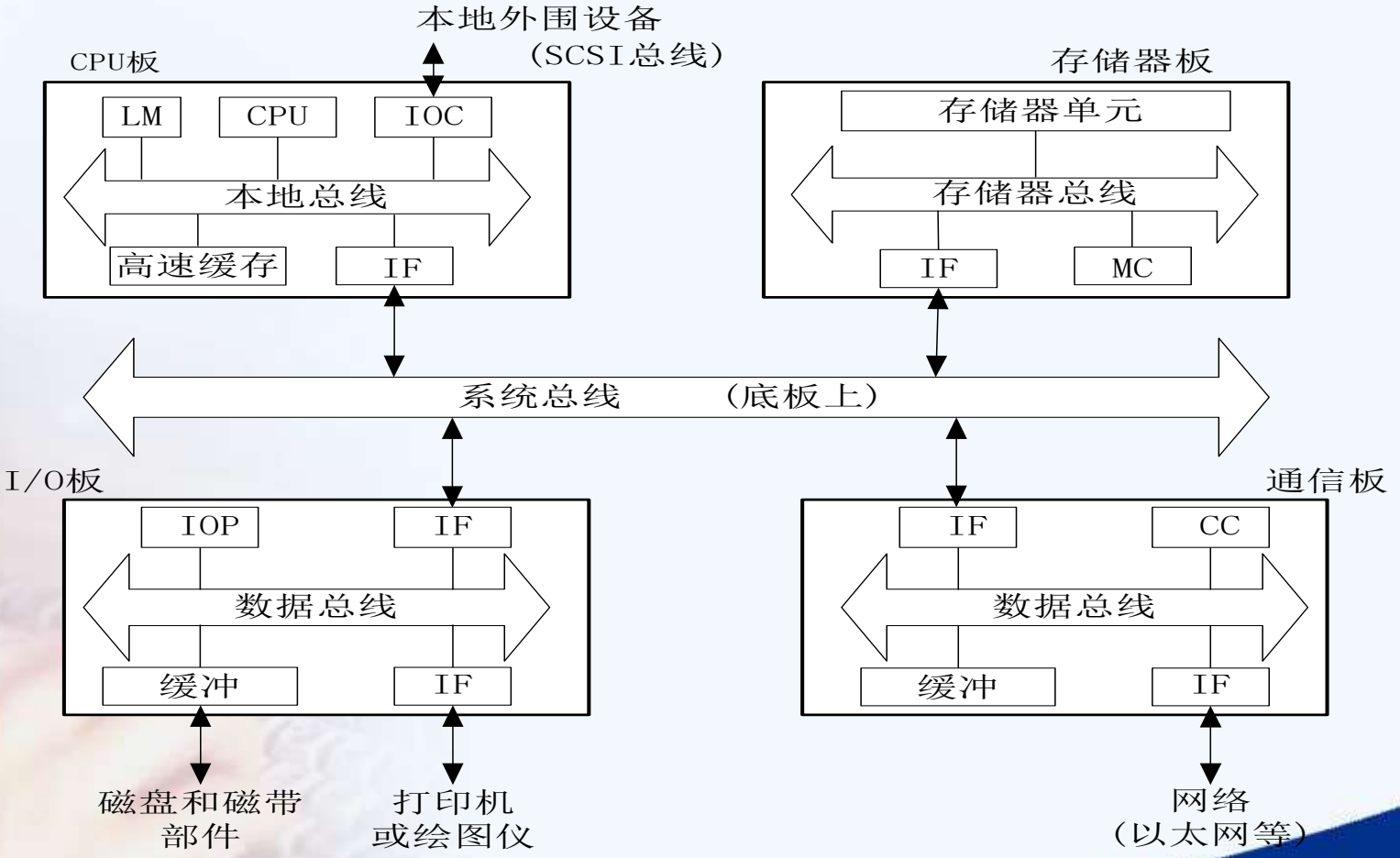


静态互连网络特性比较

网络名称	网络规模	节点度	网络直径	对剖宽度	对称	链路数
线性阵列	N	2	$N-1$	1	非	$N-1$
环形	N	2	$\lfloor N/2 \rfloor$ （双向）	2	是	N
2-D网孔	$(\sqrt{N} \times \sqrt{N})$	4	$2(\sqrt{N}-1)$	$\sqrt{N}$	非	$2(N-\sqrt{N})$
Illiac网孔	$(\sqrt{N} \times \sqrt{N})$	4	$\sqrt{N}-1$	$2\sqrt{N}$	非	$2N$
2-D环绕	$(\sqrt{N} \times \sqrt{N})$	4	$2\lfloor \sqrt{N}/2 \rfloor$	$2\sqrt{N}$	是	$2N$
二叉树	N	3	$2(\lceil \log N \rceil - 1)$	1	非	$N-1$
星形	N	$N-1$	2	$\lfloor N/2 \rfloor$	非	$N-1$
超立方	$N=2^n$	n	n	$N/2$	是	$nN/2$
立方环	$N=k\cdot 2^k$	3	$2k-1+\lfloor k/2 \rfloor$	$N/(2k)$	是	$3N/2$

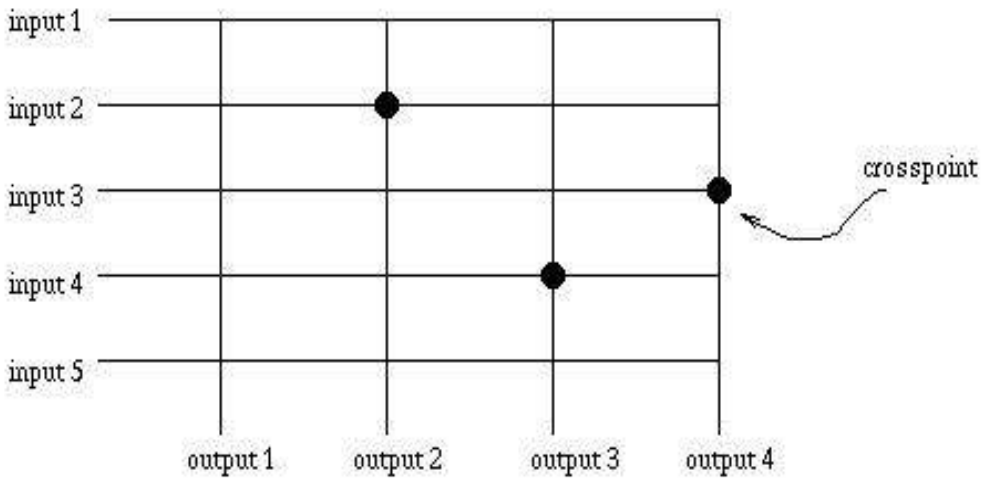
动态互连网络 (1)

- 总线：PCI、VME、Multics、Sbus、MicroChannel
  - 多处理机总线系统的主要问题包括总线仲裁、中断处理、协议转换、快速同步、高速缓存一致性协议、分事务、总线桥和层次总线扩展等



动态互连网络（2）

- 交叉开关（Crossbar）：
  - 单级交换网络，可为每个端口提供更高的带宽。象电话交换机一样，交叉点开关可由程序控制动态设置其处于“开”或“关”状态，而能提供所有（源、目的）对之间的动态连接。
  - 交叉开关一般有两种使用方式：一种是用于对称的多处理机或多计算机机群中的处理器间的通信；另一种是用于SMP服务器或向量超级计算机中处理器和存储器之间的存取。



A Small Crossbar Switch

动态互连网络（3）

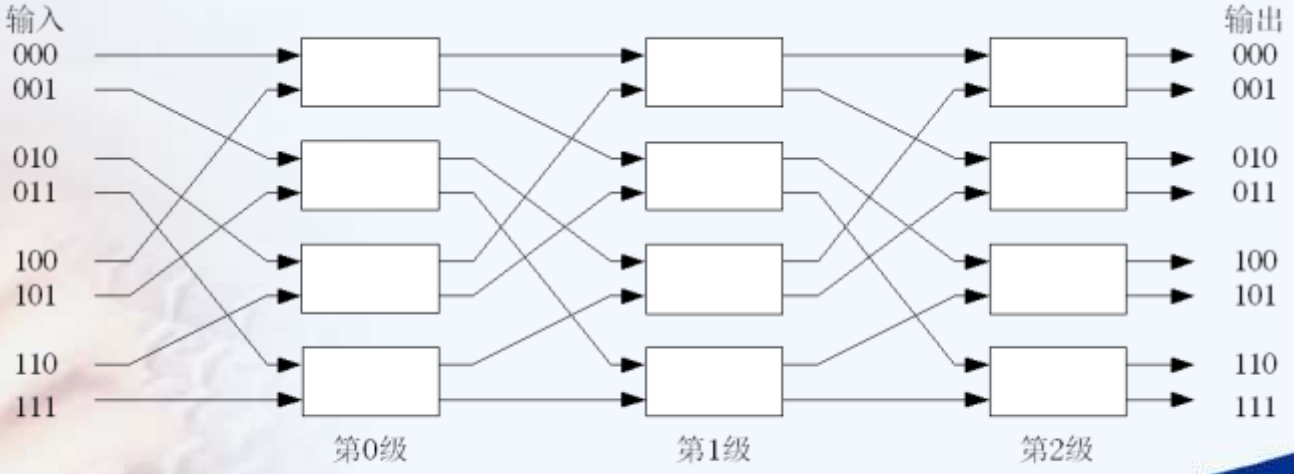
- 交换开关模块：
  - 一个交换开关模块有n个输入和n个输出，每个输入可连接到任意输出端口，但只允许一对一或一对多的映射，不允许多对一的映射，因为这将发生输出冲突



(a) 4种可能的开关连接

动态互联网络（4）

- 单级交叉开关级联起来形成多级互连网络MIN（Multistage Interconnection Network）
- 级间互连（Interstage Connection）：
  - 均匀洗牌、蝶网、多路均匀洗牌、交叉开关、立方连接
  - n输入的Ω网络需要$\log_2 n$级 2×2开关，在Ilinois大学的Cedar[2]多处理机系统中采用了Ω网络
  - Cray Y/MP多级网络，该网络用来支持8个向量处理器和256个存储器模块之间的数据传输。网络能够避免8个处理器同时进行存储器存取时的冲突。



(b) 一种8输入的Omega网络

动态互连网络比较

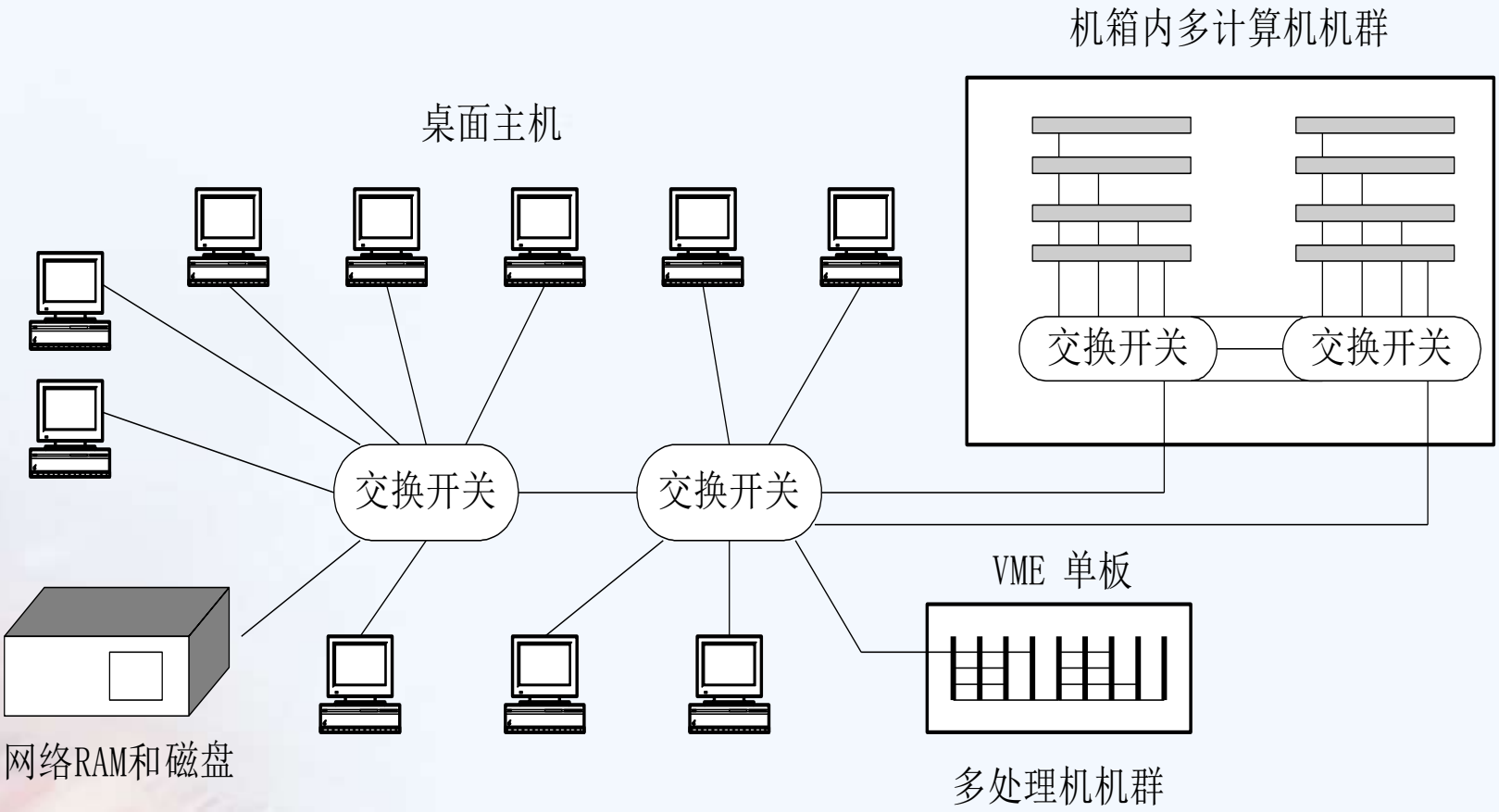
动态互连网络的复杂度和带宽性能一览表			
网络特性	总线系统	多级互连网络	交叉开关
硬件复杂度	$O(n+w)$	$O((n\log_k n)w)$	$O(n^2 w)$
每个处理器带宽	$O(wf/n) \sim O(wf)$	$O(wf)$	$O(wf)$
报道的聚集带宽	**SunFire服务器中的Gigaplane总线：2.67GB/s**	**IBM SP2中的512节点的HPS：10.24GB/s**	**Digital的千兆开关：3.4GB/s**

- n,节点规模　　w，数据宽度

标准互联网络（1）

- Myrinet:
  - Myrinet是由Myricom公司设计的千兆位包交换网络，其目的是为了构筑计算机机群，使系统互连成为一种商业产品。
  - Myrinet是基于加州理工学院开发的多计算机和VLSI技术以及在南加州大学开发的ATOMIC/LAN技术。Myrinet能假设任意拓扑结构，不必限定为开关网孔或任何规则的结构。
  - Myrinet在数据链路层具有可变长的包格式，对每条链路施行流控制和错误控制，并使用切通选路法以及定制的可编程的主机接口。在物理层上，Myrinet网使用全双工SAN链路，最长可达3米，峰值速率为（1.28＋1.28）Gbps（目前有2.56+2.56)
  - Myrinet交换开关 :8,12,16端口
  - Myrinet主机接口：32位的称作LANai芯片的用户定制的VLSI处理器，它带有Myrinet接口、包接口、DMA引擎和快速静态随机存取存储器SRAM。
  - 140 of the November 2002 TOP500 use Myrinet, including 15 of the top 100

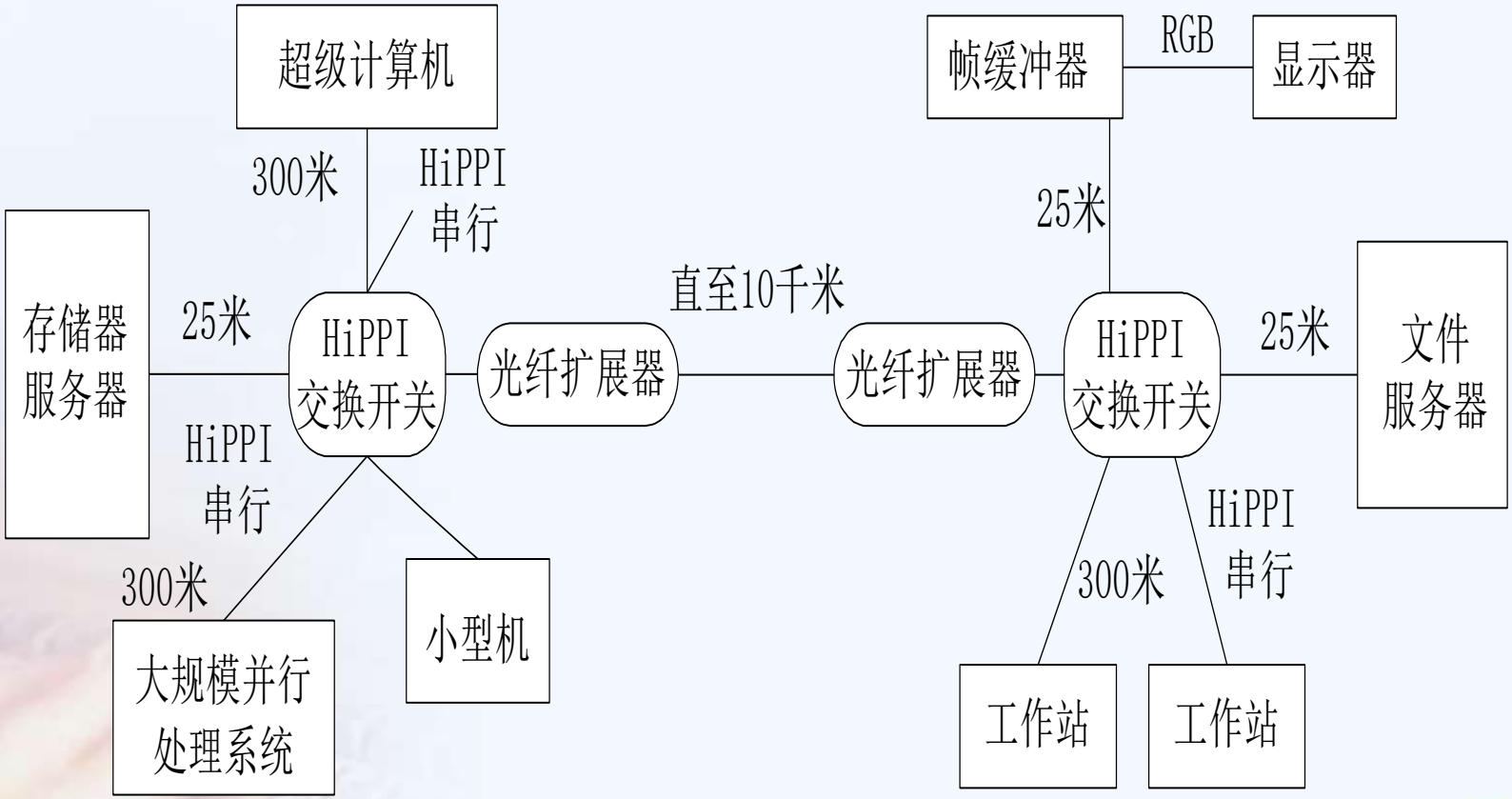
Myrinet连接的LAN/Cluster



标准互连网络（2）

- 高性能并行接口（HiPPI）
  - Los Alamos国家实验室于1987年提出的一个标准，其目的是试图统一来自不同产商生产的所有大型机和超级计算机的接口。在大型机和超级计算机工业界，HiPPI作为短距离的系统到系统以及系统到外设连接的高速I/O通道。
  - 1993年，ANSI X3T9.3委员会认可了HiPPI标准，它覆盖了物理和数据链路层，但在这两层之上的任何规定却取决于用户。
  - HiPPI是个单工的点到点的数据传输接口，其速率可达800Mbps到1.6Gbps。
  - 开发成功了一种能提供潜在的6.4Gbps速率，比HiPPI快8倍且有很低时延的超级HiPPI技术，
  - SGI公司和Los Alamos国家实验室都开发了用来构筑速率高达25.6Gbps的HiPPI交换开关的HiPPI技术。
  - HiPPI通道和HiPPI交换开关被用在SGI Power Challenge服务器、IBM 390主机、Cray Y/MP、C90和T3D/T3E等系统

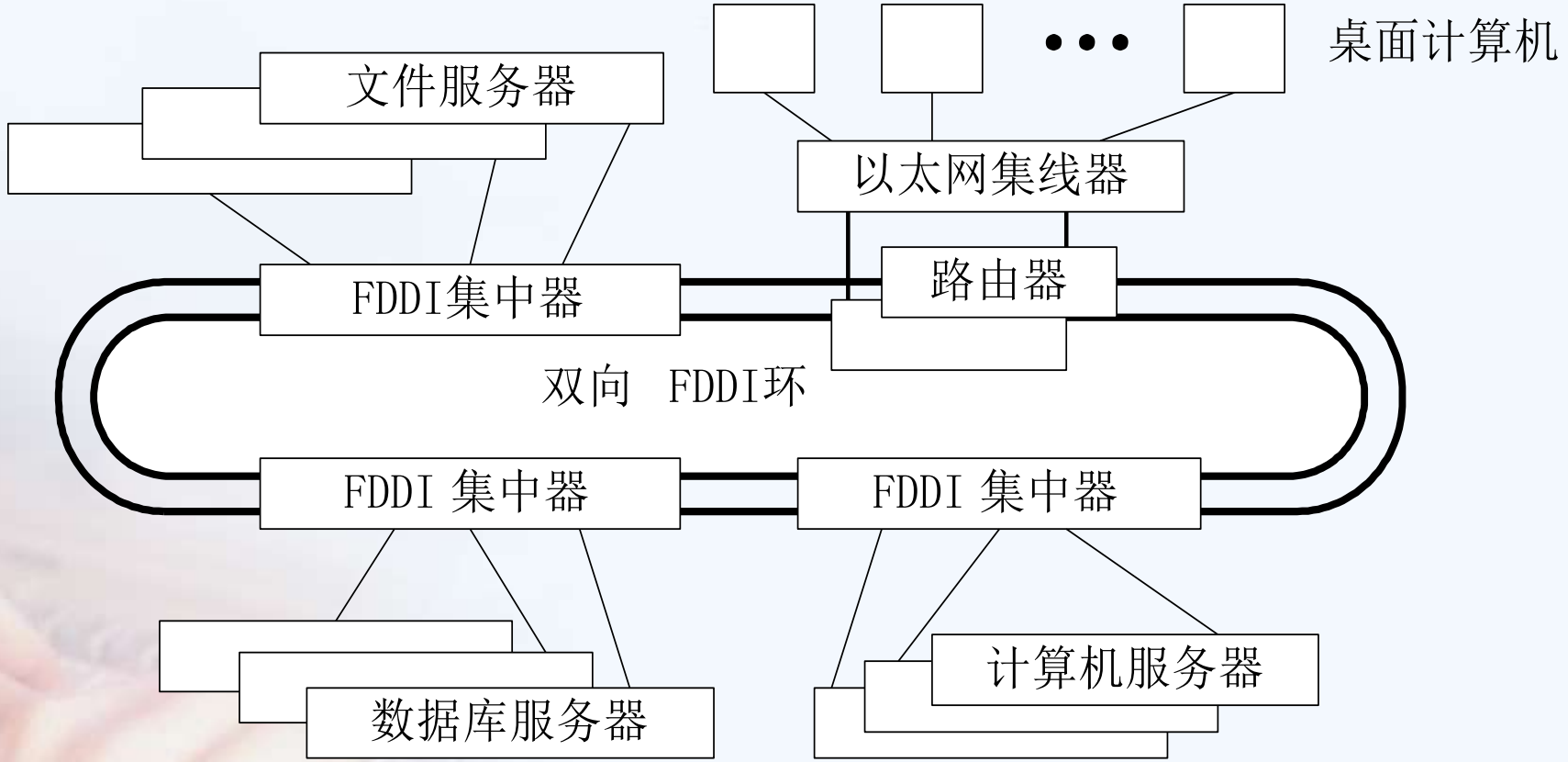
使用HiPPI通道和开关构筑的LAN主干网



标准互连网络（3）

- 光纤通道FC（Fiber Channel）：
  - 通道和网络标准的集成
  - 光纤通道既可以是共享介质，也可以是一种交换技术
  - 光纤通道操作速度范围可从100到133、200、400和800Mbps。FCSI厂商也正在推出未来具有更高速度（1、2或4Gbps）的光纤通道
  - 光纤通道的价值已被现在的某些千兆位局域网所证实，这些局域网就是基于光纤通道技术的
  - 连网拓扑结构的灵活性是光纤通道的主要财富，它支持点到点、仲裁环及交换光纤连接
- FDDI：
  - 光纤分布式数据接口FDDI（Fiber Distributed Data Interface）
  - FDDI采用双向光纤令牌环可提供100-200Mbps数据传输速率
  - FDDI具有互连大量设备的能力
  - 传统的FDDI仅以异步方式操作

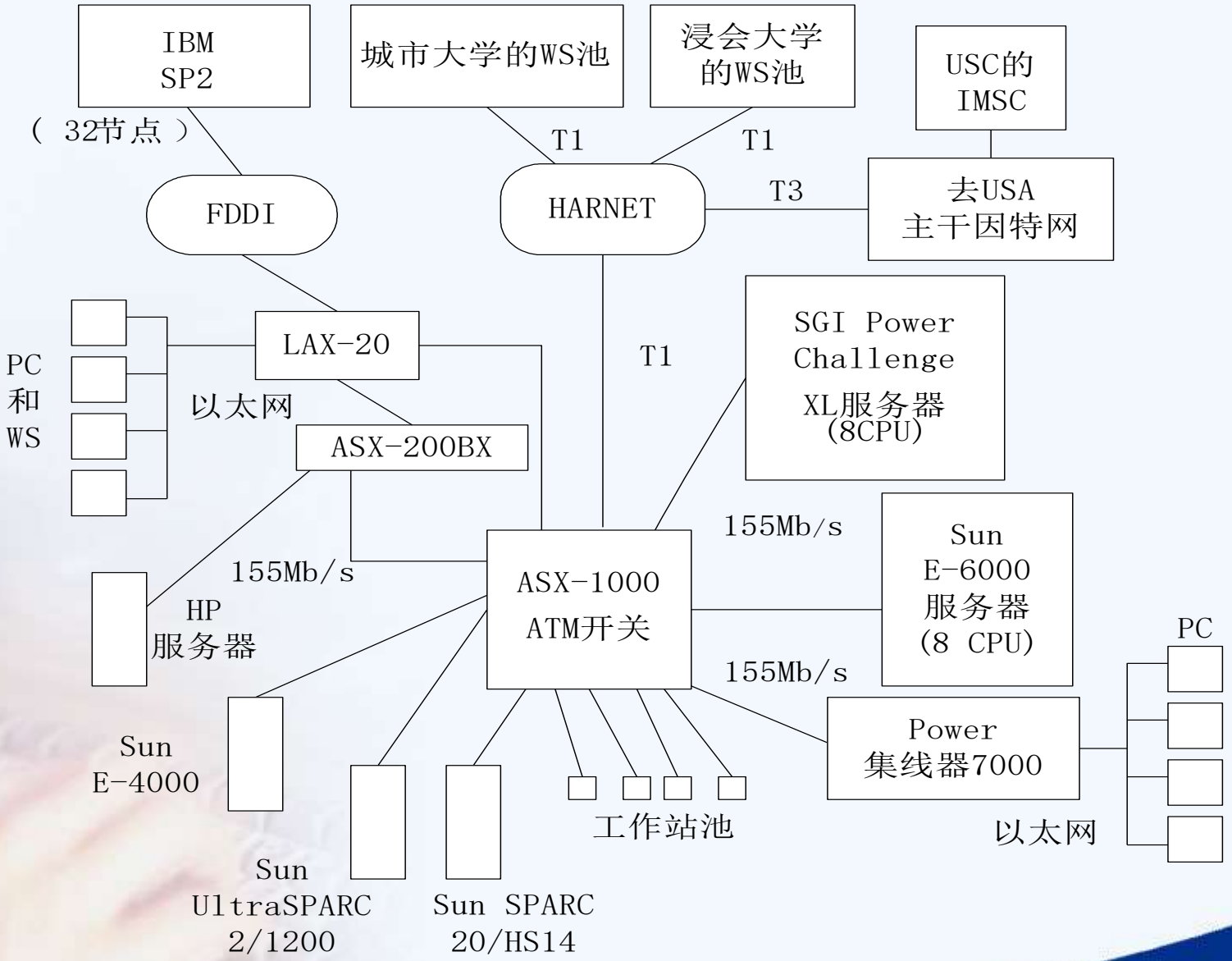
双向FDDI环作为主干网



标准互联网络（4）

- ATM（Asynchronous Transfer Mode）：
  - 由成立于1991年的ATM论坛和ITU标准定义。
  - ATM是一种独立于介质的消息传输协议，它将消息段变成更短的固定长度为53字节的报元进行传输。
  - 这种技术是基于报元交换机制。ATM的目的是将实时和突发数据的传输合并成单一的网络技术。
  - ATM网络支持从25到51、155和622Mbps不同的速率，其速率越低ATM交换器和使用的链路价格越低。

香港大学开发的Pearl机群



标准互连网络（5）

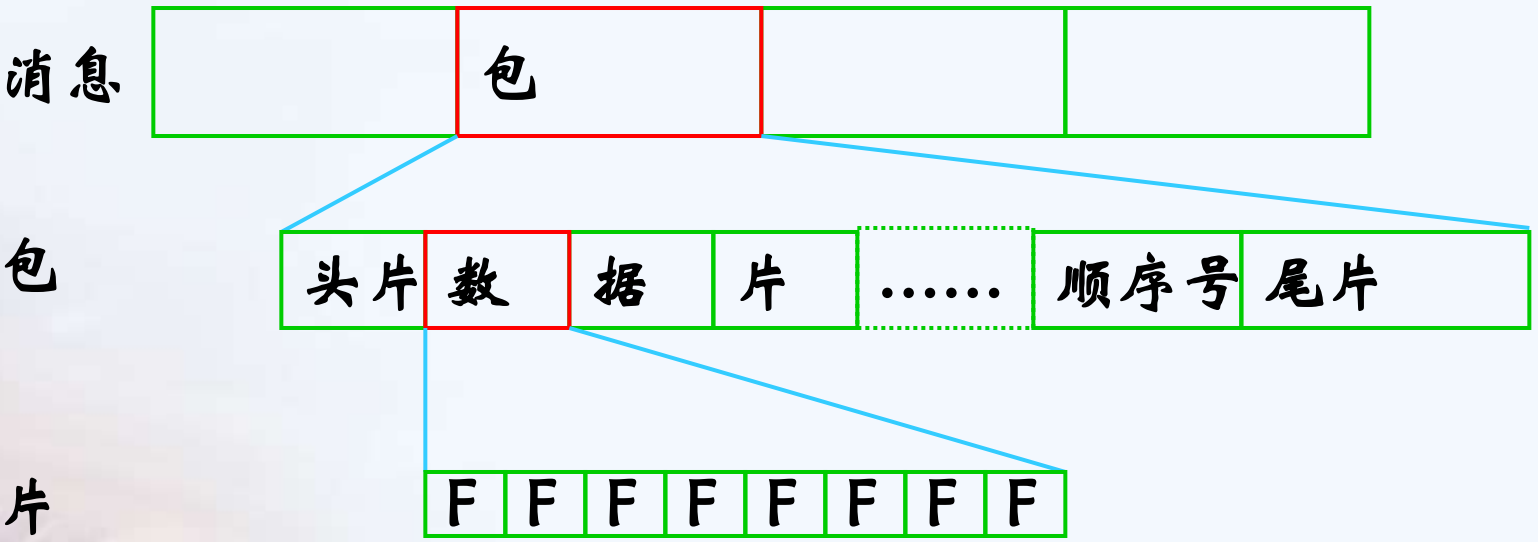
代别 类型		以太网 10BaseT	快速以太网 100BaseT	千兆位以太网 1GB
引入年代		1982	1994	1997
速度（带宽）		10Mb/s	100Mb/s	1Gb/s
最大距离	UTR（非屏蔽双扭对）	100m	100m	25－100m
	STP（屏蔽双扭对） 同轴电缆	500m	100m	25－100m
	多模光纤	2Km	412m（半双工） 2Km（全双工）	500m
	单模光纤	25Km	20Km	3Km
主要应用领域		文件共享， 打印机共享	COW计算， C/S结构， 大型数据库存取等	大型图像文件， 多媒体， 因特网， 内部网， 数据仓库等

选路方法与开关技术

- 选路(Routing)
  - 又称为选径或路由。产生消息从发源地到目的地所取的路径，要求具有较低通讯延迟、无死锁和容错能力。应用于网络或并行机上的信息交换。
- 消息、信包、片
  - 消息(Message)：是在多计算机系统的处理接点之间传递包含数据和同步消息的信息包。它是一种逻辑单位，可由任意数量的包构成。
  - 包(Packet)：包的长度随协议不同而不同，它是信息传送的最小单位，64-512位。
  - 片(Flit)：片的长度固定，一般为8位。

选路方法与开关技术

- 消息、信包、片的相互关系



选路方法与开关技术

- 信包
  - 存储－转发（*Store and Forward*）
  - 虫孔（*Wormhole*）
- 线路：用交换机

- 静态：在选路开始时所有的信 息都已到达网络
- 动态：信包可在任意时刻到达 网络

- 联机(*online*)：没有事先计算好的路径
- 脱机(*offline*)：事先算好传输路径

- 一到一（单播）
- 一到一（置换）：每个处理器开始时最多 发送一条信包，每条信包有且仅有一个 目的地；
- 多到一（集中）
- 一到多（多播）
- 一到所有（广播、组播）
- 多到多（会议）

选路方法

- 选路算法的三种机制
  - 基于算术的：开关中具有简单的算术运算功能，如维序选路；
  - 基于源地址的：在源点时就将沿路径的各个开关的输出端口地址$p_0,p_1,\cdots,p_n$包在信包的头部，每个开关只是对信包头的输出端口地址进行剥离；
  - 基于查表的：开关中含有一个选路表，对信包头中的选路域查出输出端口地址。

选路方法

- 分类
  - 最短路径/非最短路径(贪心选路/随机选路),
    如维序选路是贪心的，二阶段维序选路是随机的
  - 确定选路/自适应选路(寻径确定/寻径视网络状况)
- 维序选路(Dimension-Ordered Routing):
  一种确定的最短路径选路
  - 二维网孔中的维序选路: X-Y选路
  - 超立方中的维序选路: E-立方选路

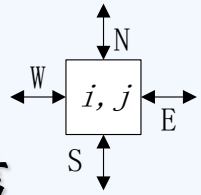
选路方法

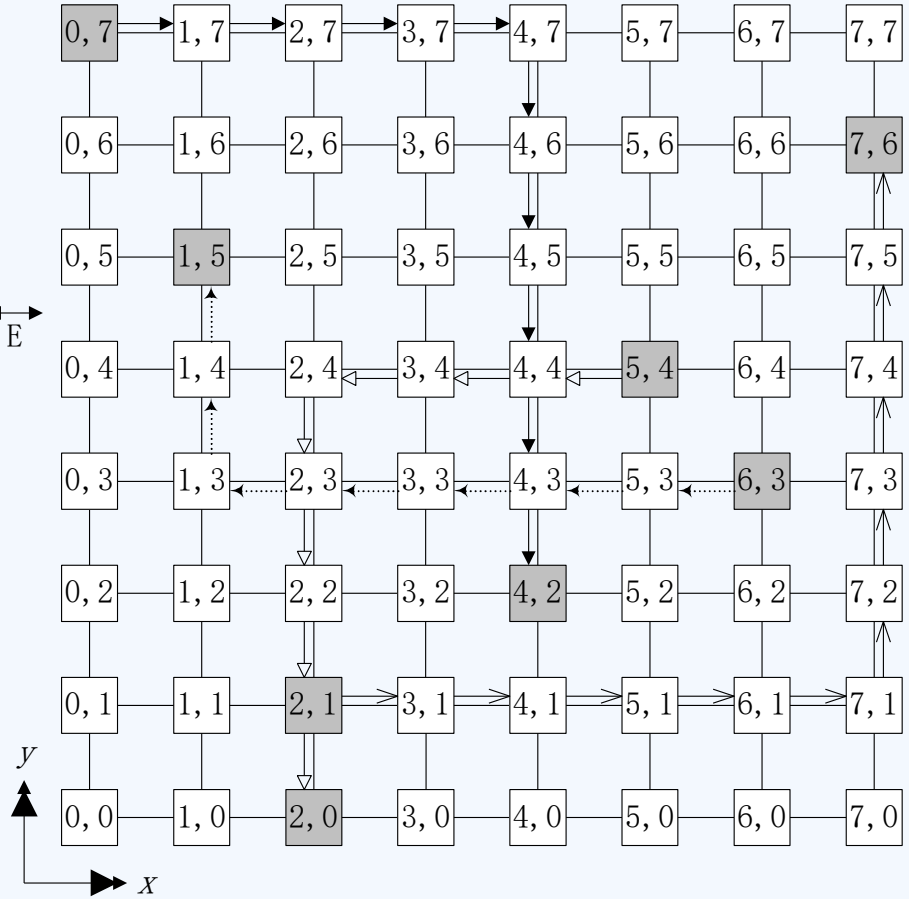
- 二维网孔上X-Y选路算法
  - begin

    step1: 沿X方向将信包送至目的地处理器所在的列

    step2: 沿Y方向将信包送至目的地处理器所在的行

    end





4(源;目的)对: (2, 1; 7, 6) → (5, 4; 2, 0) ⇢

(0, 7; 4, 2) → (6, 3; 1, 5) ⇢

图8.1

选路方法

- E-立方选路算法
  - 路由计算：　$s_{n-1}s_{n-2}\cdots s_1s_0$(源地址)

    异或 $\oplus$ $d_{n-1}d_{n-2}\cdots d_1d_0$(目的地址)

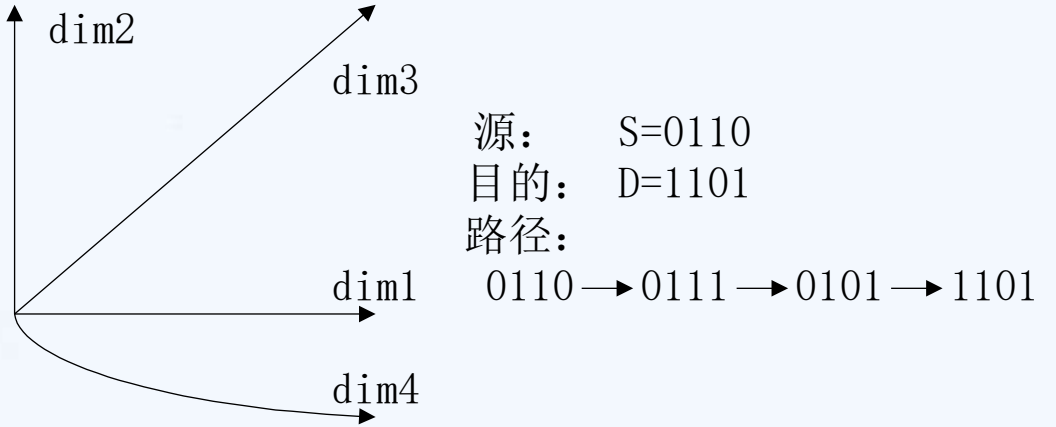
    $r_{n-1}r_{n-2}\cdots r_1r_0$(路由值)
  - 路由过程：

$s_{n-1}s_{n-2}\cdots s_1s_0 \rightarrow s_{n-1}s_{n-2}\cdots s_1s_0\oplus r_0 \rightarrow$

$s_{n-1}s_{n-2}\cdots s_1s_0\oplus r_1 \rightarrow \cdots$　($r_i=0$ 时跳过)

选路方法

- 0110(S)
  1101(D)
  1011(R)



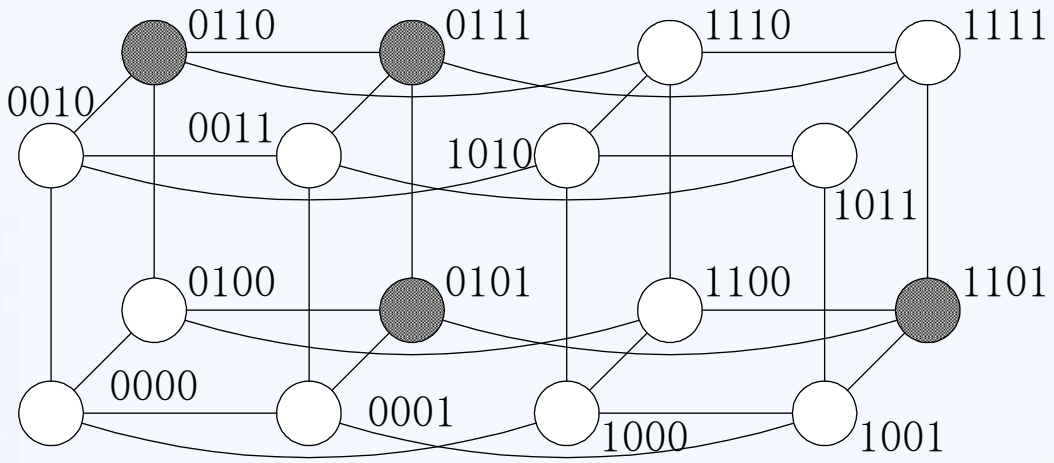


图8.2

开关技术

- 一些术语
  - 信道带宽b：每个信道有w位宽和信号传输率f = 1/t (t是时钟周期)，b = wf bits/sec
  - 节点和开关的度：与节点和开关相连的信道数目
  - 路径：信包在网络中走过的开关和链路(link)序列
  - 路由长度或距离：路由路径中包括的链路(link)数目
- 信包传输性能参数
  - 启动时间t_s(startup time)：准备包头信息等
  - 节点延迟时间t_h(per-hop time)：包头穿越相邻节点的时间
  - 字传输时间t_w(transfer time)：传输每个字的时间
  - 链路数l、信包大小m

开关技术

- 存储转发(Store-and-Forward)选路
  - 消息被分成基本的传输单位----信包(Packet)，每个信包都含有寻径信息；
  - 当一个信包到达中间节点A时，A把整个信包放入其通信缓冲器中，然后在选路算法的控制下选择下一个相邻节点B，当从A到B的通道空闲并且B的通信缓冲器可用时，把信包从A发向B；
  - 信包的传输时间：$t_{comm}(SF) = t_s + (mt_w + t_h)l = O(ml)$

  缺点：
  - 每个结点必须对整个消息和信包进行缓冲，缓冲器较大；
  - 网络时延与发送消息所经历的节点数成正比

开关技术

- 切通(Cut Through)选路
  - 在传递一个消息之前，就为它建立一条从源结点到目的结点的物理通道。在传递的全部过程中，线路的每一段都被占用，当消息的尾部经过网络后，整条物理链路才被废弃。
  - 传输时间: $t_{comm}(CT) = t_s + mt_w + lt_h = O(m+l)$

缺点：

- 物理通道非共享
- 传输过程中物理通道一直被占用

开关技术

- 虫孔(Wormhole)选路
  - Dally于1986年提出，利用了前二种方法的优点，减少了缓冲区，提高了物理通道的利用。
  - 首先把一个消息分成许多很小的片，消息的头片包含了这个消息的所有寻径信息。尾片是一个其最后包含了消息结束符的片。中间的片均为数据片；
  - 片是最小信息单位。每个结点上只需要缓冲一个片就能满足要求；
  - 用一个头片直接牵引一条从输入链路到输出链路的路径的方法来进行操作。每个消息中的片以流水的方式在网络中向前“蠕动”。每个片相当于Worm的一个节，“蠕动”以节为单位顺序地向前爬行。当消息的尾片向前“蠕动”一步后，它刚才所占用的结点就被放弃了。

开关技术

- 虫孔(Wormhole)选路

  优点：

  (1)每个结点的缓冲器的需求量小，易于用VLSI实现；

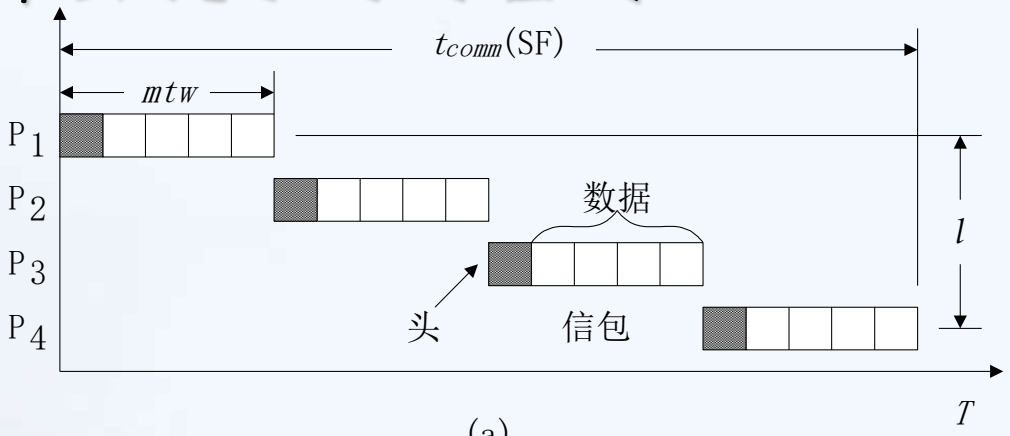
  (2)较低的网络传输延迟。存储转发传输延迟基本上正比于消息在网络中传输的距离；Wormhole与线路开关的网络传输延迟正比于消息包的长度，传输距离对它的影响很小（消息包较长时的情况）；

  (3)通道共享性好、利用率高；

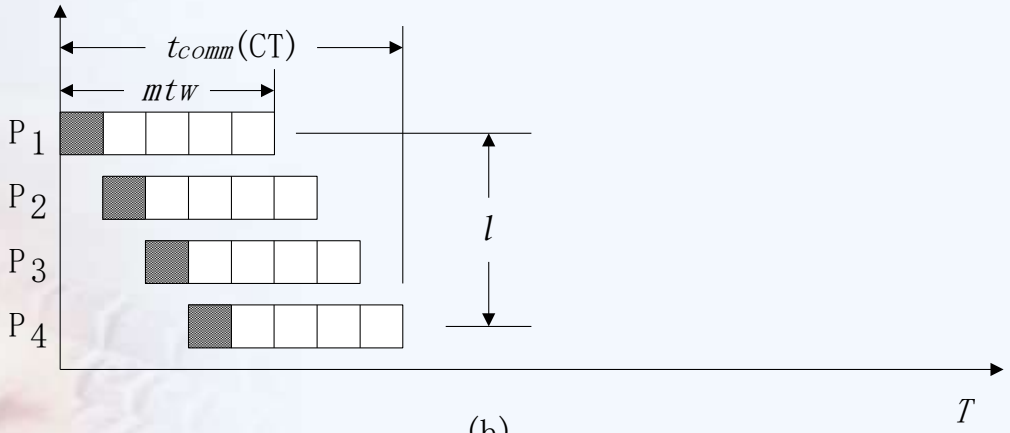
  (4)易于实现Multicast和Broadcast。

开关技术

- 几种开关技术的时空图



(a)



(b)

图8.4

单一信包一到一传输

- 距离l的计算：对于p个处理器
  - 一维环形：$l \le \lfloor p/2 \rfloor$
  - 带环绕Mesh($\sqrt{p} \times \sqrt{p}$)：$l \le 2\lfloor \sqrt{p}/2 \rfloor$
  - 超立方：$l \le \log p$
- t_{comm}(SF)的计算($t_{comm}(SF) = t_s + (mt_w + t_h)l$)
  - 一维环形：$t_{comm}(SF) = t_s + t_w \cdot m \cdot \lfloor p/2 \rfloor$
  - 带环绕Mesh：$t_{comm}(SF) = t_s + 2t_w \cdot m \cdot \lfloor \sqrt{p}/2 \rfloor$　　$t_s >> t_h$
  - 超立方：$t_{comm}(SF) = t_s + t_w \cdot m \cdot \log p$
- t_{comm}(CT)的计算($t_{comm}(CT) = t_s + mt_w + lt_h$)

$$t_{comm}(CT) = t_s + mt_w$$

- 如果m>>p：$t_{comm}(SF) \approx t_{comm}(CT) = t_s + mt_w$

一到多播送—SF模式

- 环
  - 步骤：①先左右邻近传送;②再左右二个方向同时播送
  - 示例：

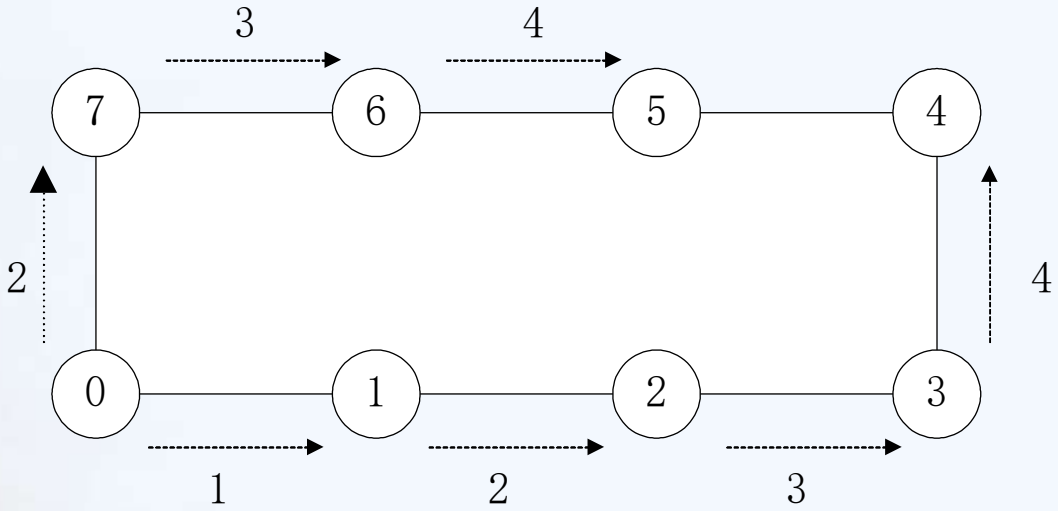


图8.5

- 通讯时间：$t_{one-to-all}(SF) = (t_s + mt_w)\lceil p/2 \rceil$

一到多播送—SF模式

- 环绕网孔
  - 步骤：①先完成一行中的播送;②再同时进行各列的播送
  - 示例：
    共4步(2步行、2步列)

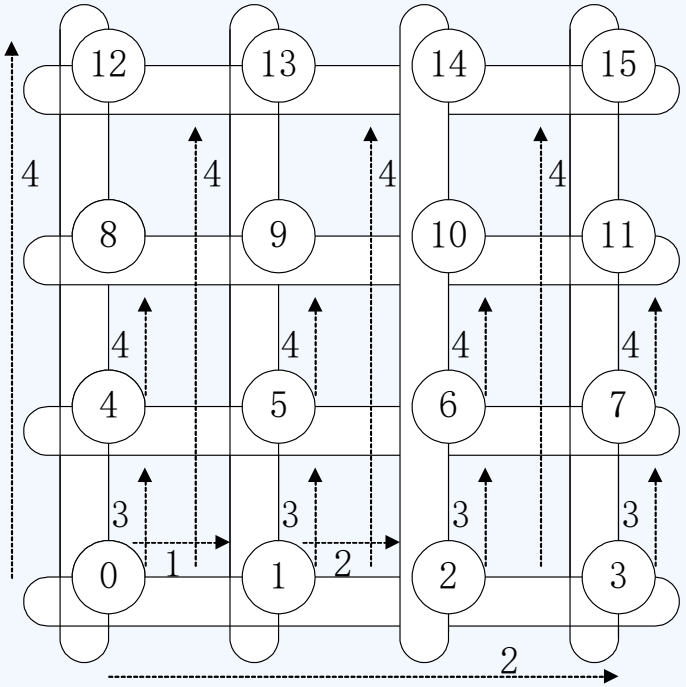
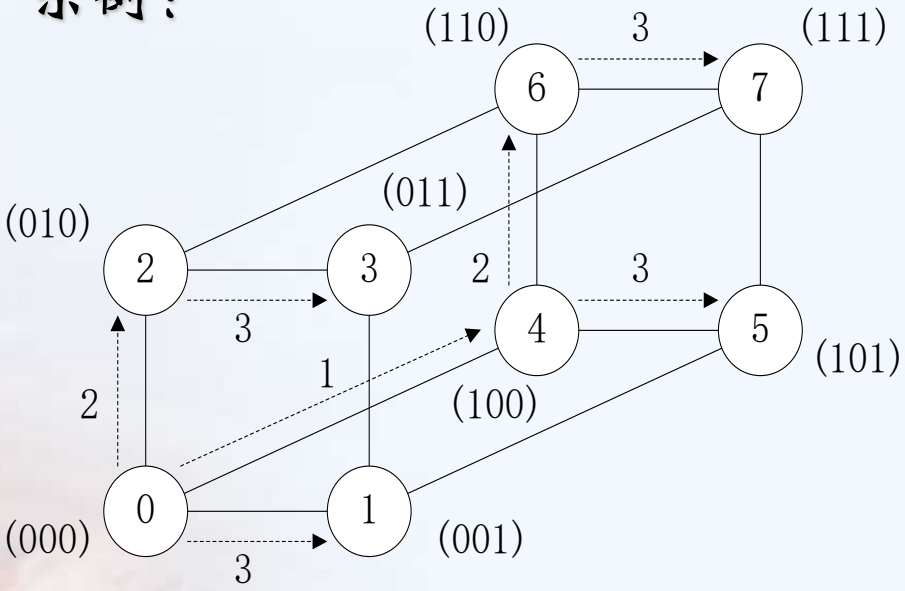


图 8.6

- 通讯时间：$t_{one-to-all}(SF) = 2(t_s + mt_w)\left\lceil \frac{\sqrt{p}}{2} \right\rceil$

一到多播送—SF模式

- 超立方
  - 步骤：从低维到高维，依次进行播送；
  - 示例：



  - 通讯时间：$t_{one-to-all}(SF) = (t_s + mt_w)\log p$

一到多播送—CT模式

- 环
  - 步骤：

(1)先发送至p/2远的处理器；

(2)再同时发送至$p/2^2$远的处理器；

……

(i)再同时发送至$p/2^i$远的处理器；

  - 示例：图8.8

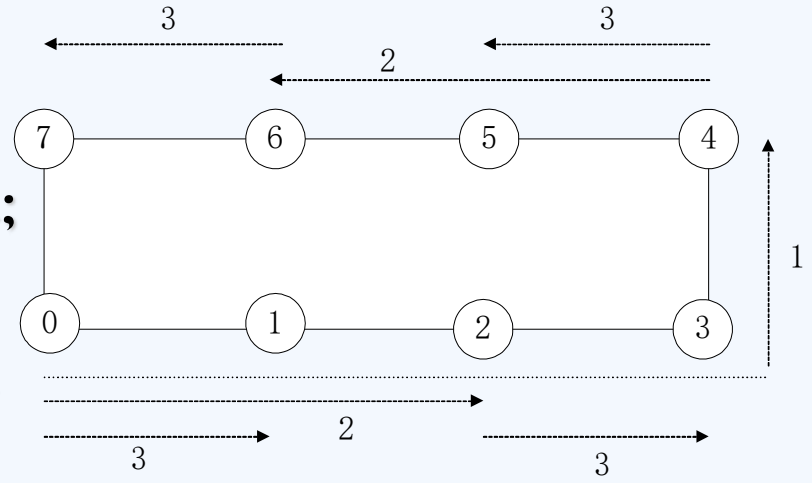


图 8.8

- 通讯时间：

$$t_{one-to-all}(CT) \overset{(8.2)}{=} \sum_{i=1}^{\log p}(t_s + mt_w + t_h p/2^i)$$

$$= t_s \log p + mt_w \log p + t_h(p-1)$$

$$\approx (t_s + mt_w)\log p \qquad (t_h\text{可忽略时})$$

一到多播送—CT模式

- 网孔
  - 步骤：

(1)先进行行播送；

$$// \; t_s \log\sqrt{p} + mt_w \log\sqrt{p} + t_h(\sqrt{p}-1)$$

(2)再同时进行列播送；

$$// \; t_s \log\sqrt{p} + mt_w \log\sqrt{p} + t_h(\sqrt{p}-1)$$

  - 示例： 图8.9

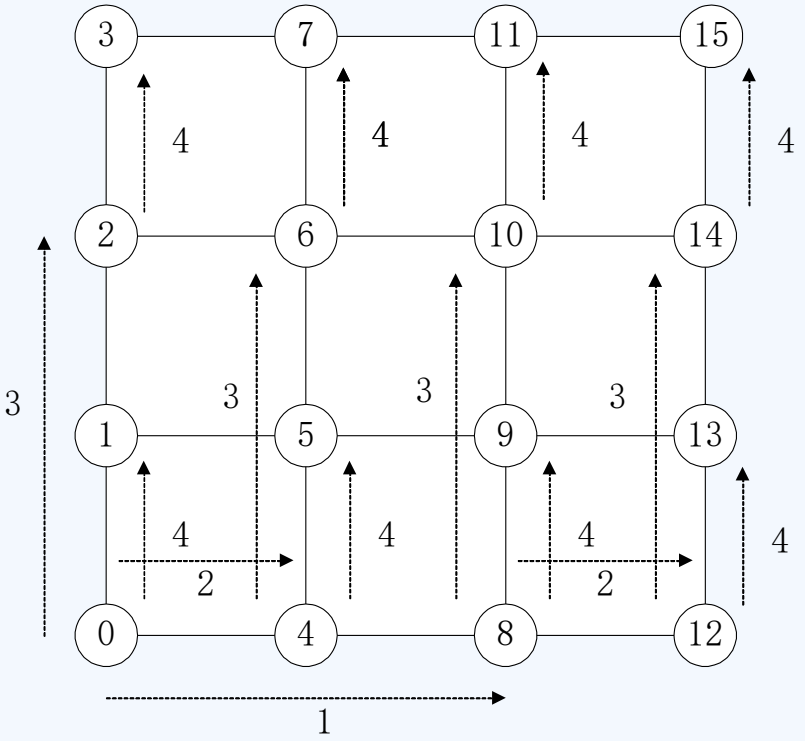


图 8.9

  - 通讯时间：

$$t_{one-to-all}(CT) = 2(t_s \log\sqrt{p} + mt_w \log\sqrt{p} + t_h(\sqrt{p}-1))$$
$$= (t_s + mt_w)\log p + 2t_h(\sqrt{p}-1)$$

一到多播送—CT模式

- 超立方
  - 步骤：
    依次从低维到高维播送，d-立方，d=0,1,2,3,4…；
  - 通讯时间：$t_{one-to-all}(CT) = (t_s + mt_w)\log p$

多到多播送—SF模式

- 环
  - 步骤：

同时向右(或左)播送刚接收到的信包

  - 示例： 图8.10
  - 通讯时间：

$$t_{all-to-all}(SF) = (t_s + mt_w)(p-1)$$

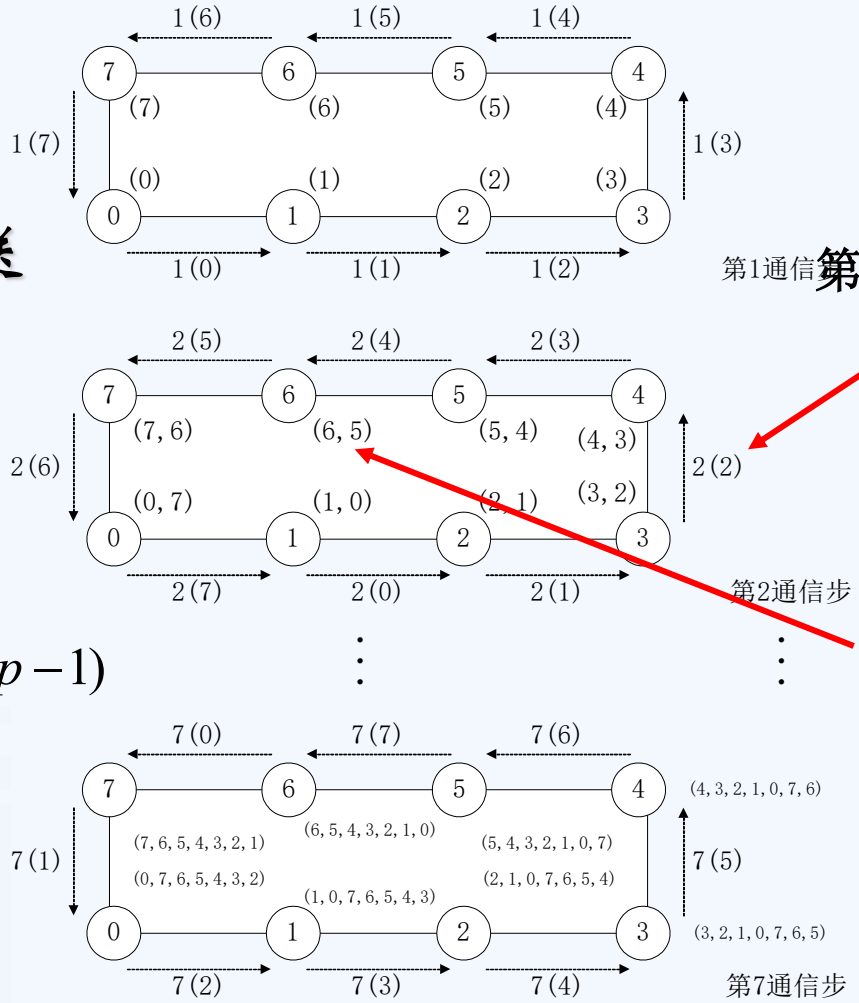


图8. 10

多到多播送—SF模式

- 环绕网孔
  - 步骤：

(1)先进行行的播送；

(2)再进行列的播送；

  - 示例： 图8.11

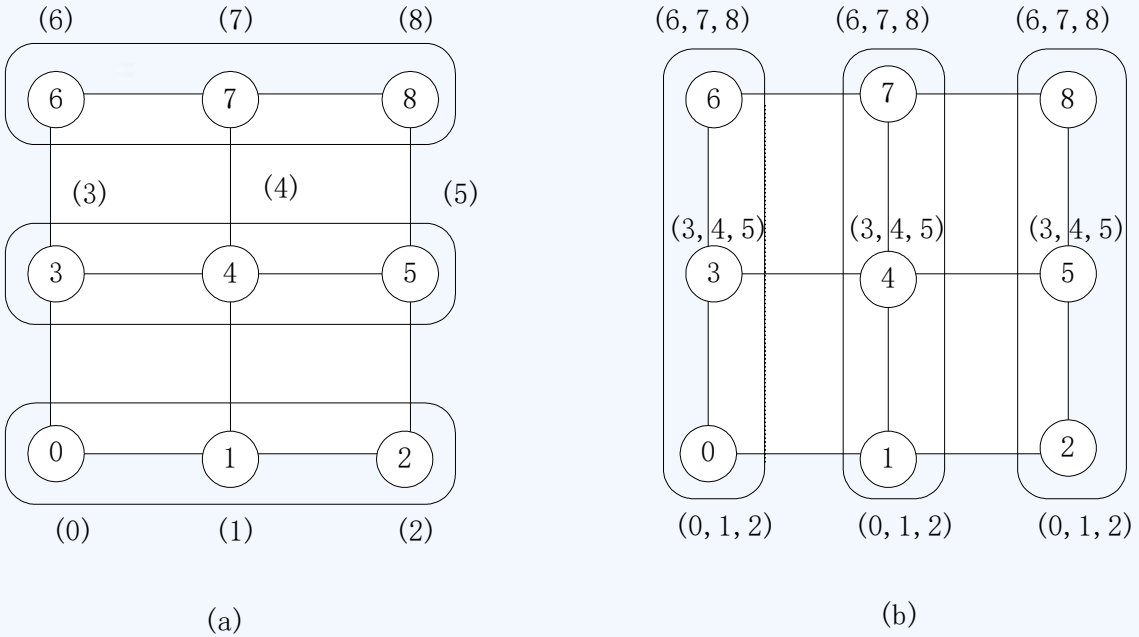


图8. 11

- 通讯时间：

$$t_{all-to-all}(SF) = (t_s + mt_w)(\sqrt{p} - 1) + (t_s + m\sqrt{p} \cdot t_w)(\sqrt{p} - 1)$$
$$= 2t_s(\sqrt{p} - 1) + mt_w(p - 1)$$

多到多播送—SF模式

- 超立方
  - 步骤：
    依次按维进行
    多到多的播送；
  - 示例： 图8.12
  - 通讯时间：

$$t_{all-to-all}(SF) = \sum_{i=1}^{\log p}(t_s + 2^{i-1}mt_w)$$
$$= t_s \log p + mt_w(p-1)$$

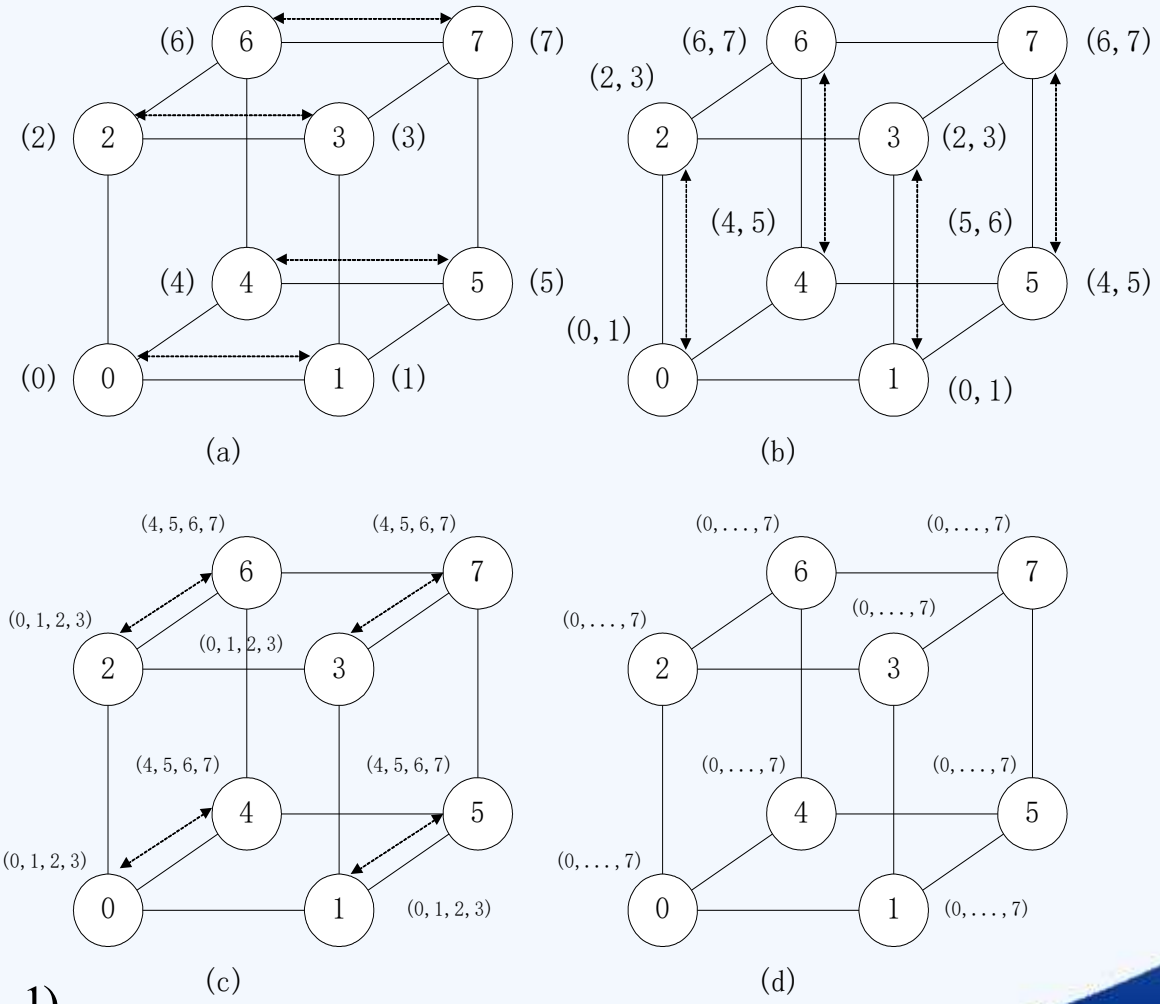


图8. 12

多到多播送—CT模式

- $t_{all-to-all}(CT) = t_{all-to-all}(SF)$
- 使用一到多的策略会造成链路竞争

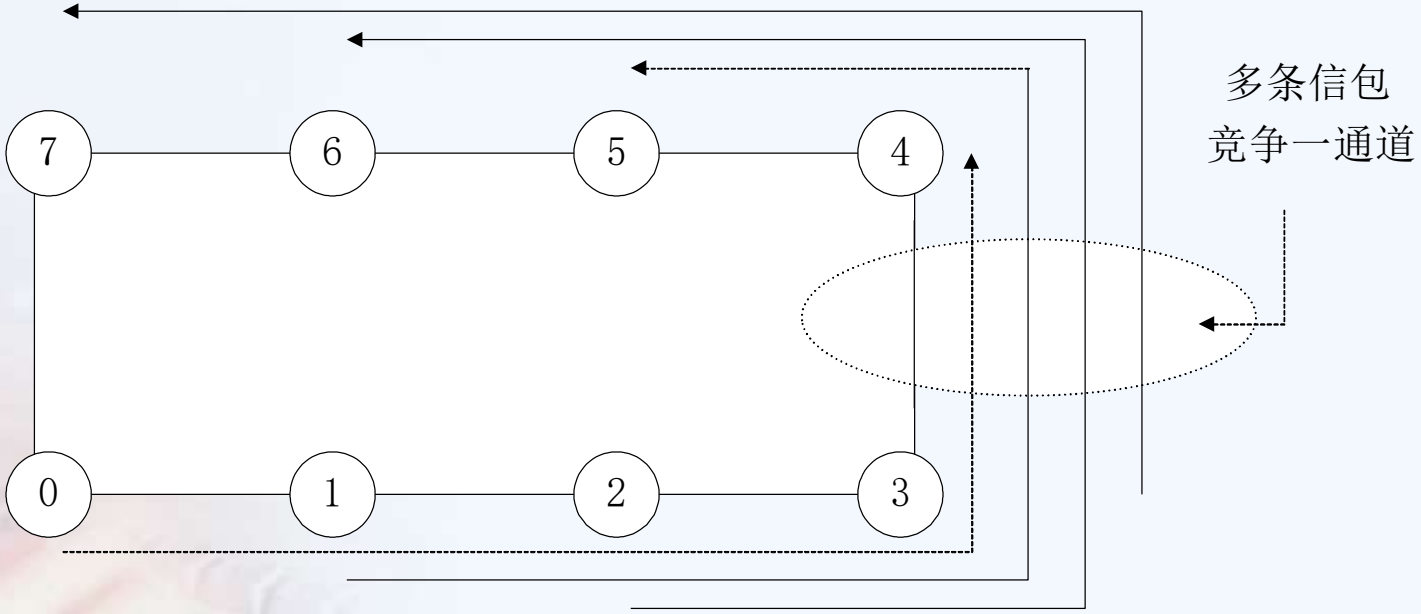


图8.13

3.并行计算性能评测

CPU的某些基本性能指标

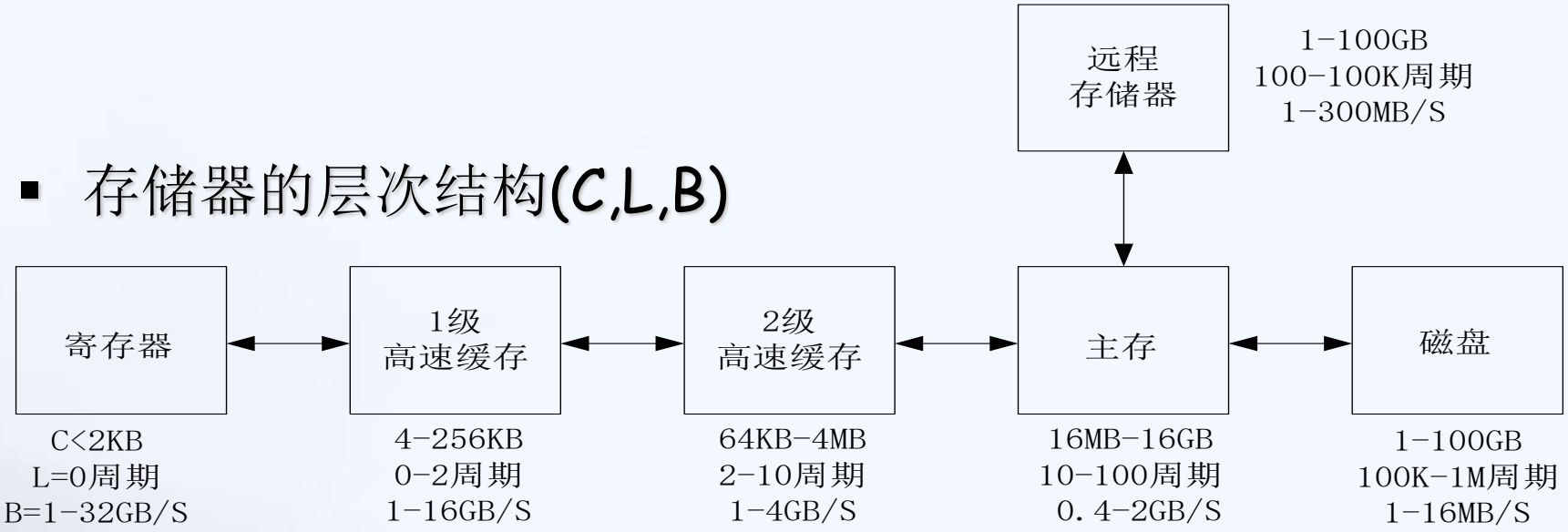
- 工作负载
  - 执行时间
  - 浮点运算数
  - 指令数目

- 并行执行时间 T_{comput} 为计算时间，T_{paro} 为并行开销时间（进程创建、切换等），T_{comm} 为相互通信时间

$$T_n = T_{comput} + T_{paro} + T_{comm}$$

存储器性能

- 存储器的层次结构(C,L,B)



- 估计存储器的带宽
- RISC add r1,r2,r3 r 8bytes 100MHz
- B = 3*8*100*10^6 B/s= 2.4GB/s

并行与通信开销

- 并行和通信开销：相对于计算很大。
- PowerPC (每个周期 15ns 执行4flops;
- 创建一个进程1.4ms 可执行372000flops)
- 开销的测量：乒--乓方法（Ping-Pong Scheme）节点0发送m个字节给节点1；节点1从节点0接收m个字节后，立即将消息发回节点0。总的时间除以2，即可得到点到点通信时间，也就是执行单一发送或接收操作的时间。
- 可一般化为热土豆法（Hot-Potato），也称为救火队法（Fire-Brigade) 0——1 —— 2 —— … —— -n-1 —— 0

Ping-Pong Scheme

- if （my _node _id =0） then /*发送者*/
- start _time =second（ ）
- send an m-byte message to node 1
- receive an m-byte message from node 1
- end_time = second（ ）
- total_time = end_time – start_time
- communication_time[i] = total_time/2
- else if （my_node_id = 1） then /*接收者*/
- receive an m-byte message from node 0
- send an m-byte message to node 0
- endif

并行开销的表达式：点到点通信

- 通信开销 $t(m) = t_0 + m/r_\infty$
- 通信启动时间 t_0
- 渐近带宽r_∞：传送无限长的消息时的通信速率
- 半峰值长度$m_{1/2}$：达到一半渐近带宽所要的消息长度
- 特定性能π_0：表示短消息带宽

$$t_0 = m_{1/2} / r_\infty = 1 / \pi_0$$

并行开销的表达式：整体通信

- 典型的整体通信有：
  - 播送（Broadcasting）：处理器0发送m个字节给所有的n个处理器
  - 收集（Gather）：处理0接收所有n个处理器发来在消息，所以处理器0最终接收了m n个字节；
  - 散射（Scatter）：处理器0发送了m个字节的不同消息给所有n个处理器，因此处理器0最终发送了m n个字节；
  - 全交换（Total Exchange）：每个处理器均彼此相互发送m个字节的不同消息给对方，所以总通信量为mn^2个字节；
  - 循环移位（Circular-shift）：处理器i发送m个字节给处理器i+1，处理器n-1发送m个字节给处理器0，所以通信量为m n个字节。

机器的成本、价格与性/价比

- 机器的成本与价格
- 机器的性能/价格比 Performance/Cost Ratio：指用单位代价（通常以百万美元表示）所获取的性能（通常以MIPS或MFLOPS表示）
- 利用率（Utilization）：可达到的速度与峰值速度之比

算法级性能评测

- 加速比性能定律
  - 并行系统的加速比是指对于一个给定的应用，并行算法（或并行程序）的执行速度相对于串行算法（或串行程序）的执行速度加快了多少倍。
  - Amdahl 定律
  - Gustafson定律
  - Sun和Ni定律
- 可扩放性评测标准
  - 等效率度量标准
  - 等速度度量标准
  - 平均延迟度量标准

Amdahl 定律

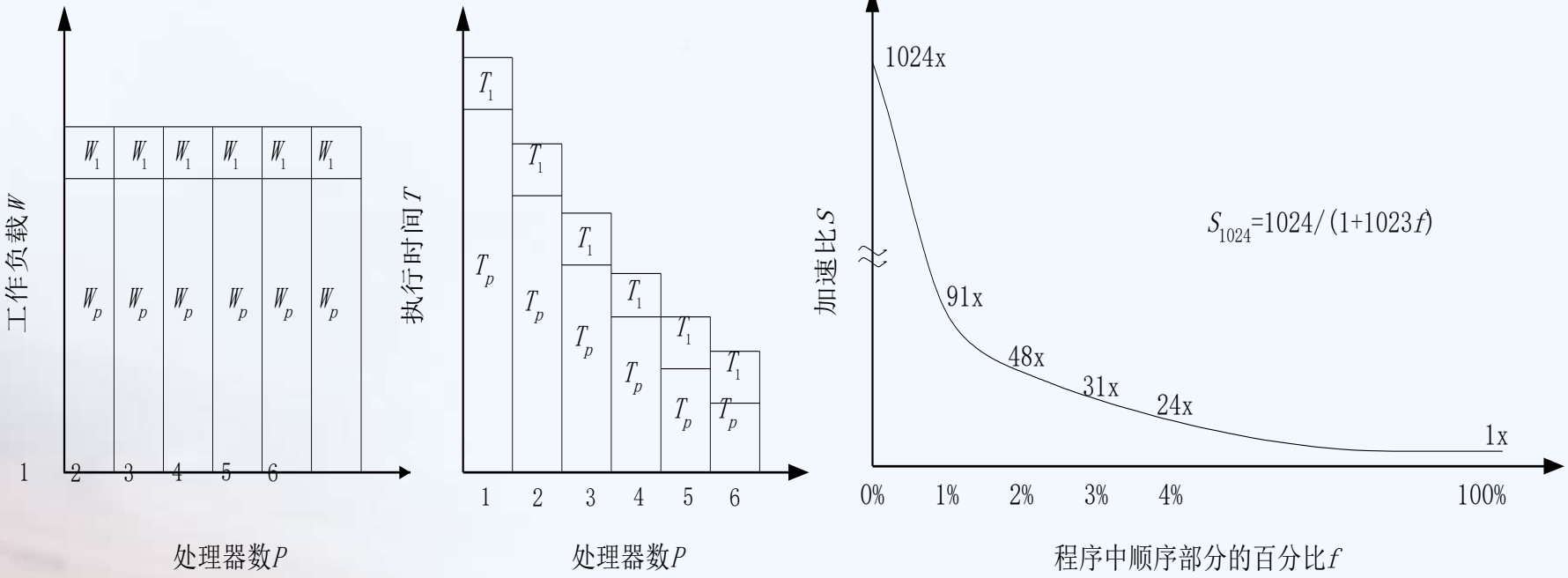
- P：处理器数；
- W：问题规模（计算负载、工作负载，给定问题的总计算量）；
  - W_s：应用程序中的串行分量，f是串行分量比例（$f = W_s/W$，$W_s=W_1$）；
  - W_p：应用程序中可并行化部分，1-f为并行分量比例；
  - $W_s + W_p = W$；
- $T_s=T_1$：串行执行时间，T_p：并行执行时间；
- S：加速比，E：效率；
- 出发点：
  - 固定不变的计算负载；
  - 固定的计算负载分布在多个处理器上的，
  - 增加处理器加快执行速度，从而达到了加速的目的。

Amdahl定律（cont'd)

- 固定负载的加速公式： $S = \dfrac{Ws + Wp}{Ws + W_P / p}$
-
- $W_s + W_p$可相应地表示为f+（1-f）
-
$$S = \frac{f + (1-f)}{f + \dfrac{1-f}{p}} = \frac{p}{1 + f(p-1)}$$
- $p \to \infty$时，上式极限为： S= 1 / f
- W_o为额外开销

$$S = \frac{W_S + W_P}{W_S + \dfrac{W_P}{p} + W_O} = \frac{W}{fW + \dfrac{W(1-f)}{p} + W_O} = \frac{p}{1 + f(p-1) + W_O p / W}$$

Amdahl定律 (cont'd)



Gustafson定律

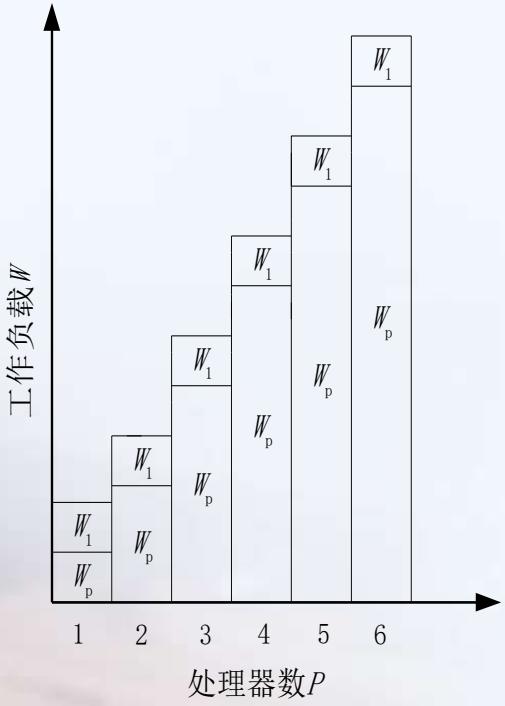
- 出发点：
  - 对于很多大型计算，精度要求很高，即在此类应用中精度是个关键因素，而计算时间是固定不变的。此时为了提高精度，必须加大计算量，相应地亦必须增多处理器数才能维持时间不变；
  - 除非学术研究，在实际应用中没有必要固定工作负载而计算程序运行在不同数目的处理器上，增多处理器必须相应地增大问题规模才有实际意义。
- Gustafson加速定律： $$S' = \frac{W_S + pWp}{W_S + p \cdot Wp / p} = \frac{W_S + pWp}{W_S + W_P}$$

$$S' = f + p\,(1\text{-}f) = p + f\,(1\text{-}p) = p\text{-}f\,(p\text{-}1)$$

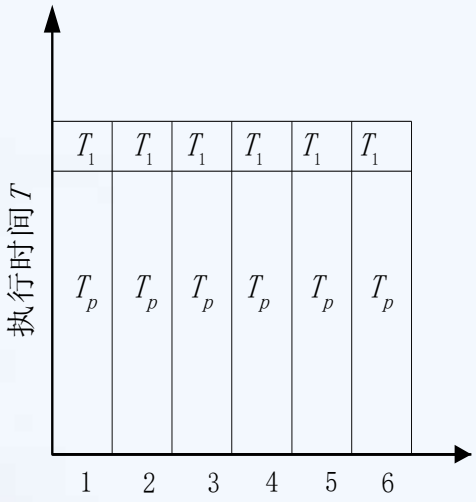
- 并行开销W_o：

$$S' = \frac{W_S + pW_P}{W_S + W_P + W_O} = \frac{f + p(1-f)}{1 + W_O / W}$$

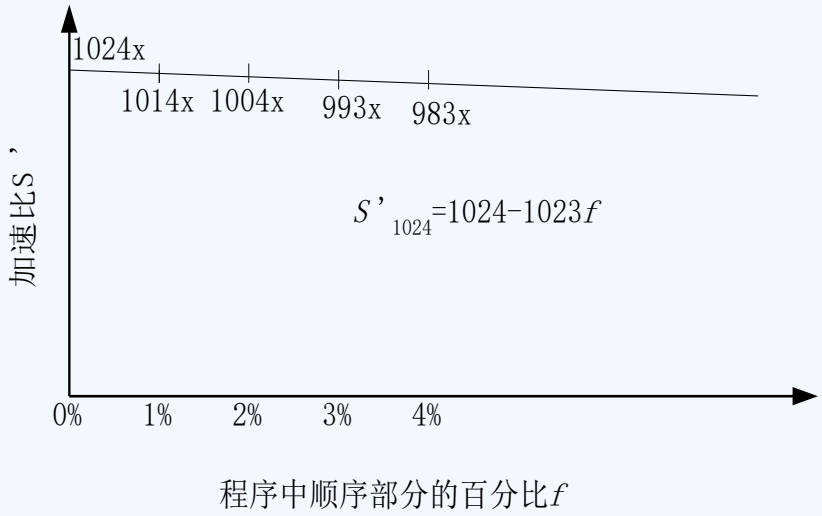
Gustafson定律（cont'd)



(a)



(b)



(c)

Sun 和 Ni 定律

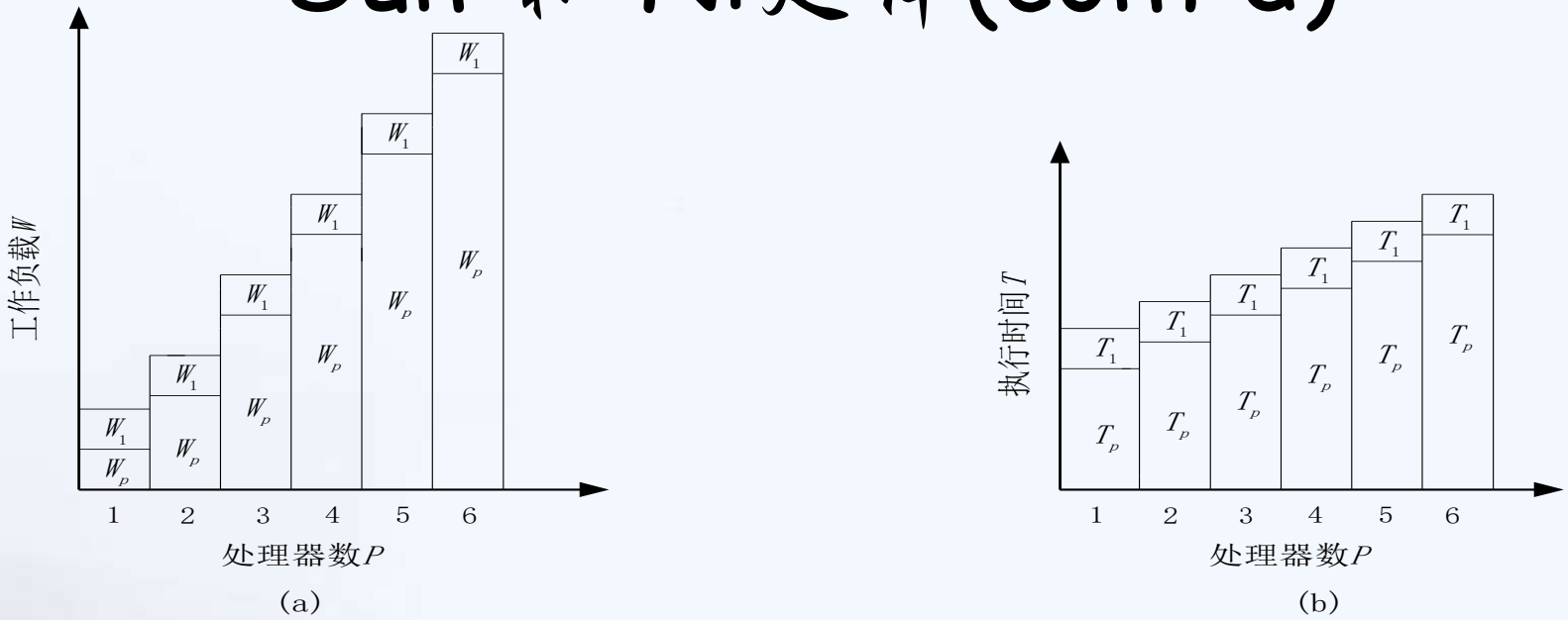
- 基本思想：
  - 只要存储空间许可，应尽量增大问题规模以产生更好和更精确的解（此时可能使执行时间略有增加）。
  - 假定在单节点上使用了全部存储容量M并在相应于W的时间内求解之，此时工作负载W= fW + （1-f）W。
  - 在p 个节点的并行系统上，能够求解较大规模的问题是因为存储容量可增加到pM。令因子G（p）反应存储容量增加到p倍时并行工作负载的增加量，所以扩大后的工作负载W = fW + （1-f）G（p）W。
- 存储受限的加速公式：

$$S'' = \frac{fW + (1-f)G(p)W}{fW + (1-f)G(p)W/p} = \frac{f + (1-f)G(p)}{f + (1-f)G(p)/p}$$

- 并行开销W_o:

$$S' = \frac{fW + (1-f)WG(p)}{fW + (1-f)G(p)W/p + W_O} = \frac{f + (1-f)G(p)}{f + (1-f)G(p)/p + W_O/W}$$

Sun 和 Ni定律(cont'd)



- G（p）=1时就是Amdahl加速定律；
- G（p）=p 变为 f + p（1-f），就是Gustafson加速定律
- G（p）>p时，相应于计算机负载比存储要求增加得快，此时 Sun和 N i 加速比 Amdahl 加速和 Gustafson 加速均高。

加速比讨论

- 参考的加速经验公式： $p/\log p \le S \le P$
- 线性加速比：很少通信开销的矩阵相加、内积运算等
- $p/\log p$的加速比：分治类的应用问题
- 通信密集类的应用问题：$S = 1 / C$（p）
- 超线性加速 *（如不同处理器在不同分支同时搜索）*
- 绝对加速：最佳并行算法与串行算法
- 相对加速：同一算法在单机和并行机的运行时间

可扩放性评测标准

- 并行计算的可扩放性（Scalability）也是主要性能指标
  - 可扩放性最简朴的含意是在确定的应用背景下，计算机系统（或算法或程序等）性能随处理器数的增加而按比例提高的能力
- 影响加速比的因素：处理器数与问题规模
  - 求解问题中的串行分量
  - 并行处理所引起的额外开销（通信、等待、竞争、冗余操作和同步等）
  - 加大的处理器数超过了算法中的并发程度
- 增加问题的规模有利于提高加速的因素：
  - 较大的问题规模可提供较高的并发度；
  - 额外开销的增加可能慢于有效计算的增加；
  - 算法中的串行分量比例不是固定不变的（串行部分所占的比例随着问题规模的增大而缩小）。
- 增加处理器数会增大额外开销和降低处理器利用率，所以对于一个特定的并行系统（算法或程序），它们能否有效利用不断增加的处理器的能力应是受限的，而度量这种能力就是可扩放性这一指标。

可扩放性评测标准(cont'd)

- 可扩放性:调整什么和按什么比例调整
  - 并行计算要调整的是处理数p和问题规模W，
  - 两者可按不同比例进行调整，此比例关系（可能是线性的，多项式的或指数的等）就反映了可扩放的程度。
- 并行算法和体系结构
- 可扩放性研究的主要目的：
  - 确定解决某类问题用何种并行算法与何种并行体系结构的组合，可以有效地利用大量的处理器；
  - 对于运行于某种体系结构的并行机上的某种算法当移植到大规模处理机上后运行的性能；
  - 对固定的问题规模，确定在某类并行机上最优的处理器数与可获得的最大的加速比；
  - 用于指导改进并行算法和并行机体系结构，以使并行算法尽可能地充分利用可扩充的大量处理器
- 目前无一个公认的、标准的和被普遍接受的严格定义和评判它的标准

等效率度量标准

- 令t_{ie} 和t_{io} 分别是并行系统上第i个处理器的有用计算时间和额外开销时间（包括通信、同步和空闲等待时间等）

$$T_e = \sum_{i=0}^{p-1} t_e^i = Ts \qquad T_o = \sum_{i=0}^{p-1} t_o^i$$

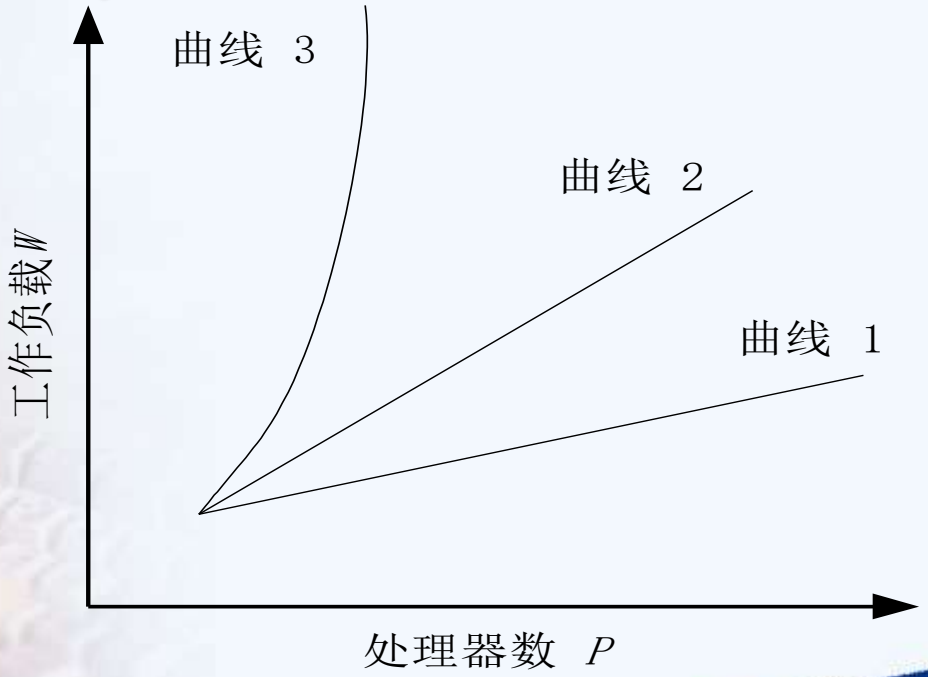
- T_p 是p个处理器系统上并行算法的运行时间，对于任意i显然有$T_p = t_{ie} + t_{io}$，且 $T_e + T_o = pT_p$
- 问题的规模W为最佳串行算法所完成的计算量$W=T_e$

$$S = \frac{T_e}{T_p} = \frac{T_e}{\frac{T_e + T_o}{p}} = \frac{p}{1 + \frac{T_o}{T_e}} = \frac{p}{1 + \frac{T_o}{W}} \qquad E = \frac{S}{P} = \frac{1}{1 + \frac{T_o}{T_e}} = \frac{1}{1 + \frac{T_o}{W}}$$

- 如果问题规模W保持不变，处理器数p增加，开销T_o增大，效率E下降。为了维持一定的效率（介于0与1之间），当处理数p增大时，需要相应地增大问题规模W的值。由此定义函数$f_E(p)$为问题规模W随处理器数p变化的函数，为等效率函数（ISO-efficiency Function）（Kumar1987）

等效率度量标准（cont'd)

- 曲线1表示算法具有很好的扩放性；曲线2表示算法是可扩放的；曲线 3表示算法是不可扩放的。
- 优点：简单可定量计算的、少量的参数计算等效率函数
- 缺点：如果T_o无法计算出（在共享存储并行机中）



等速度度量标准

- p 表示处理器个数，W表示要求解问题的工作量或称问题规模（在此可指浮点操作个数），T为并行执行时间，定义并行计算的速度V为工作量W除以并行时间T
- p个处理器的并行系统的平均速度定义为并行速度V除以处理器个数 p:

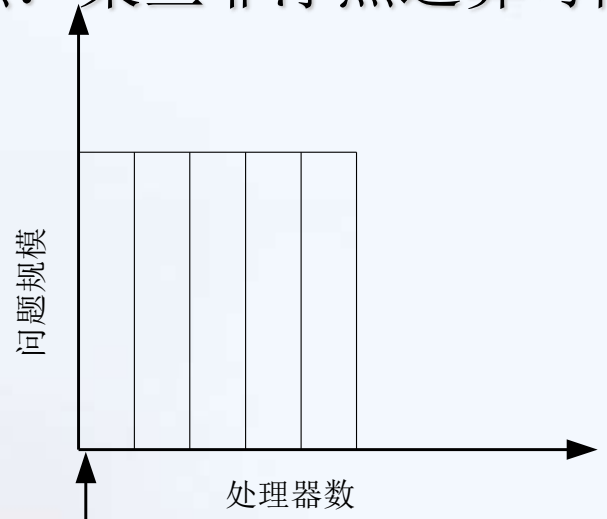
$$\overline{V} = \frac{V}{p} = \frac{W}{pT}$$

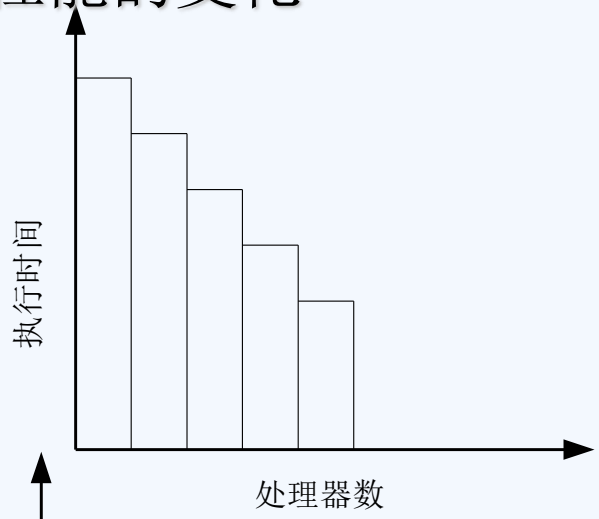
- W是使用p个处理器时算法的工作量，令W'表示当处理数从p增大到p'时，为了保持整个系统的平均速度不变所需执行的工作量，则可得到处理器数从p到p'时平均速度可扩放度量标准公式

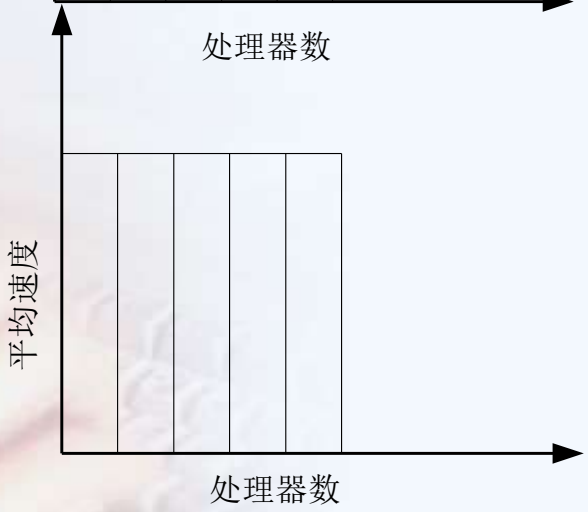
$$\Psi(p,p') = \frac{W/p}{W'/p'} = \frac{p'W}{pW'}$$

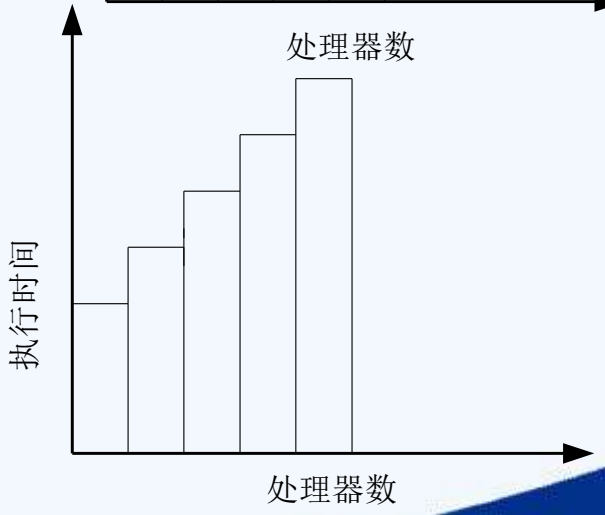
等速度度量标准（cont'd）

- 优点：直观地使用易测量的机器性能速度指标来度量
- 缺点：某些非浮点运算可能造成性能的变化









平均延迟度量标准

- T_i为P_i的执行时间（包括延迟L_i），Pi的总延迟时间为“L_i+启动时间+停止时间”。定义系统平均延迟时间为

$$\bar{L}(W,p)=\sum_{i=1}^{p}\left(T_{para}-T_i+L_i\right)\Big/p$$

- $pT_{para}=T_o+T_s$ $\qquad T_o=p\bar{L}(W,p)$

$$\bar{L}(W,p)=T_{para}-T_{seq}\,/\,p$$

- $\bar{L}(W,p)$ 在p个处理器上求解工作量为W问题的平均延迟
- $\bar{L}(W',p')$在p’个处理器上求解工作量为W’问题的平均延迟当处理器数由p变到p’，而推持并行执行效率不变，则定义平均延迟可扩放性度量标准为

$$\Phi(E,p,p')=\frac{\bar{L}(W,p)}{\bar{L}(W',p')}$$

平均延迟度量标准（Cont'd)

- 优点：平均延迟能在更低层次上衡量机器的性能
- 缺点：需要特定的软硬件才能获得平均延迟

